

Personalised Nutrition Recommender

NUTRIX

Calorie Targets, Recipe Recommendations and Optimised Meal Plans Learned
from Food.com Reviews and the NHANES Survey

Complete Project Report and Study Guide

21CSC305P — Machine Learning Laboratory

All eleven lab programs applied to one real problem, delivered as a working web application

This document explains the idea, the reasoning behind every design decision, the mechanism of each stage, the exact commands used, the code, the results obtained, how to interpret them, and how the finished application is built and tested.

Name _____

Register number _____

Contents

Part	Chapter	What it covers
I	1. The Idea	Title, the real-world problem, why the original scripts were replaced, scope
	2. Core Concepts	Energy targets, macronutrients, recommender systems, offline evaluation, optimisation
II	3. The Datasets	Food.com recipes and reviews, NHANES, recipe photos, cleaning
	4. Environment Setup	Python and Node environments, packages, data download, project structure
	5. System Mechanism	The pipeline from raw files to the running application
	6. The Code Layer	Every module in src/ and what its main functions do
III	7. Program 1	Loading and viewing the datasets
	8. Program 2	Summary statistics, nutrition checks and cleaning
	9. Program 3	Linear regression of daily energy intake
	10. Program 4.1	Bayesian logistic regression with credible intervals
	11. Program 4.2	Support vector machine for a recipe's course
	12. Program 5.1	K-means nutrition archetypes
	13. Program 5.2	Gaussian mixture model of user taste
	14. Program 5.3	Hierarchical clustering of ingredients
	15. Program 6	PCA recipe map and truncated SVD of the review matrix
	16. Program 7	Hidden Markov Model of eating phases
	17. Program 8	CART rules for the archetypes
	18. Program 9	Ensemble learning for recommendation ranking
IV	19. The Meal Planner	Mixed-integer program, candidate selection, speed experiments
	20. The NUTRIX Application	Localhost or application, architecture, API, storage, every page
	21. Testing and Validation	API checks, planner checks, builds, visual review
	22. Evaluation Summary	All results in one place
	23. Limitations	What the project does not claim
	24. Future Work	Where to take it next
V	25. Reproducing Everything	Exact commands to rebuild and run every part
	26. Glossary	Every term used
	27. Expected Viva Questions	Likely questions with answers
	28. References	Sources

Part I — The Idea

1. The Idea

1.1 Project title

Personalised Nutrition Recommender (NUTRIX): Calorie Targets, Recipe Recommendations and Optimised Meal Plans Learned from Food.com Reviews and the NHANES Survey.

1.2 The problem in the real world

A person who wants to lose weight, build muscle or simply eat better has to answer two questions every day. The first is *how much*: how many calories, and how much protein, carbohydrate and fat. The second is *what*: which actual meals deliver those numbers and are also meals the person enjoys cooking and eating.

Most diet applications answer the first question with a formula and the second with a fixed list of healthy meals. The formula is rarely checked against how real people eat, and the fixed list fails for a well-known reason: people stop following plans built from food they do not like. Adherence, not the arithmetic of calories, is where diets usually break down.

Meanwhile recipe websites hold millions of reviews that record what people actually cook. If those reviews can be turned into a model of taste, and the model's suggestions can be fitted to a person's targets, the result is a plan that is both correct and likely to be followed. That is what this project builds.

1.3 Why the original scripts were replaced

The project started from two Colab scripts (*untitled0.py* and *untitled1.py*) intended as a nutrition recommender. Reviewing them showed that no result they produced could be trusted:

Problem in the original scripts	Why it breaks the project
Calories, protein, fat and carbohydrate were generated with <code>np.random</code>	Every model trained on them learns noise; nothing transfers to real food
The 'healthy' label was a hand rule on the same random features	The SVM only re-learns the rule, so its accuracy proves nothing
Required calories were generated from a linear formula plus small noise	Linear regression on that target scores R^2 near 1 by construction
<code>food.csv</code> had a 'genres' column holding movie genres	The 'foods' were most likely a renamed movie dataset
Recommendations were sorted only by average rating	Every user saw the same list; the per-user ratings were never used

Table 1. Why the starting scripts could not be developed further.

This project keeps the goal and rebuilds every part on real data, with evaluations designed so that no model can see its answer.

1.4 What the system does

1. Calculates a daily calorie and macronutrient target from age, sex, height, weight, activity and goal, and compares it with the intake that NHANES adults of the same profile actually report.
2. Learns taste from 221,732 Food.com reviews and recommends recipes a person is likely to cook, explaining each one.
3. Fits recommended recipes into day or week meal plans that hit the targets, using an optimiser.

4. Describes the recipe collection by course, nutrition archetype and a two-dimensional map.
5. Delivers all of it in NUTRIX, a web application with a meal log, progress tracking and recipe photos.

1.5 Scope

The project is	The project is not
A complete, reproducible machine-learning pipeline on public data	A medical or dietetic tool; targets come from a population equation
A working web application that runs on the user's own computer	A hosted online service with accounts (it can be deployed; see Chapter 24)
An honest evaluation, including where the models are weak	A claim that the recommender predicts exactly what someone will cook

2. Core Concepts

2.1 Energy targets

Resting energy expenditure is the energy the body uses at rest. The Mifflin-St Jeor equation (Mifflin et al., 1990) estimates it from weight in kilograms, height in centimetres and age in years. Multiplying by a physical activity level (PAL) gives total daily energy, and a goal adjustment turns that into a target:

```
resting energy (men)   = 10 x weight + 6.25 x height - 5 x age + 5
resting energy (women) = 10 x weight + 6.25 x height - 5 x age - 161
PAL:   sedentary 1.2   light 1.375   moderate 1.55   active 1.725
goal:  lose -500 kcal   maintain 0     gain +300 kcal
target = max(resting energy x PAL + goal adjustment, 1200 kcal for women or 1500 kcal for men)
```

The target is then split between protein, carbohydrate and fat by a share of energy that depends on the goal (30/40/30 to lose weight, 20/50/30 to maintain, 25/50/25 to gain), using 4 kcal per gram for protein and carbohydrate and 9 kcal per gram for fat.

2.2 Macronutrients, Atwater factors and daily values

The Atwater factors (4, 4 and 9 kcal per gram) mean a recipe's calories should roughly equal $4 \times \text{protein} + 4 \times \text{carbohydrate} + 9 \times \text{fat}$. This identity is used twice: to recover the units Food.com uses, and to detect nutrition panels that are internally inconsistent. Food.com reports every nutrient except calories as a percent of a daily value, so those values have to be converted back to grams.

2.3 Recommender systems

Explicit feedback is a rating a user chooses to give. **Implicit feedback** is an action that reveals interest. On Food.com a review means the user cooked the recipe, so every review is treated as a positive signal whatever its stars. 77% of rated reviews give five stars, so stars carry little extra information.

Production recommenders work in two stages. **Candidate generators** cheaply score the whole catalogue and keep a few hundred items; a **ranker** then orders those candidates with a richer model. This project uses four generators (popularity, trending, collaborative filtering by PureSVD, and content similarity) and an ensemble ranker. A **cold-start** item has no interaction history, so collaborative methods cannot score it, only content can.

2.4 Evaluating a recommender offline

Leave-last-out in time: for each user the most recent recipe is hidden and the model, given only earlier history, must rank it among all recipes. **HR@10** (hit rate) is the share of users whose hidden recipe is in the top ten. **NDCG@10** also rewards placing it higher, and **MRR** is the mean of $1 / \text{rank}$. A random ranking of 78,930 recipes gives $\text{HR@10} = 0.013\%$.

2.5 Mixed-integer optimisation

A mixed-integer linear program chooses values for variables, some restricted to whole numbers, to minimise a linear objective subject to linear constraints. Here binary variables decide which recipe and portion fills each meal slot, and continuous variables measure how far the day's nutrients are from target. The HiGHS solver finds a plan and can prove how close it is to the best possible plan.

Part II — Data, Setup and Mechanism

3. The Datasets

3.1 Food.com Recipes and Interactions

Collected by Majumder et al. (2019) from Food.com, one of the largest recipe-sharing sites. The 2008-2018 cut distributed with UCSD's data science courses is used, downloaded from its public Google Drive folder.

File	Rows	Columns used
RAW_recipes.csv	83,782	name, id, minutes, submitted, tags, nutrition, n_steps, steps, description, ingredients, n_ingredients
RAW_interactions.csv	731,927	user_id, recipe_id, date, rating (0-5; 0 means a review without stars)

Table 2. Food.com files.

Each recipe's *nutrition* field is a list: calories, total fat, sugar, sodium, protein, saturated fat and carbohydrate. Calories are in kcal; the rest are percent daily value. The daily values were recovered from the data itself: with the values below, the Atwater identity reproduces stated calories with a median ratio of 0.98, and sugar almost never exceeds carbohydrate.

```
DAILY_VALUE = {
  "fat_g": 65, "sugar_g": 25, "sodium_mg": 2400,
  "protein_g": 50, "sat_fat_g": 20, "carbs_g": 300,
}
```

3.2 NHANES 2017-2018

The National Health and Nutrition Examination Survey, run by the US Centers for Disease Control and Prevention, measures participants' bodies, asks about activity and records two 24-hour dietary recalls.

File	What it contributes
DEMO_J	Age, sex, pregnancy status
BMX_J	Measured weight and height
DR1TOT_J, DR2TOT_J	Day 1 and day 2 dietary recall totals, with completeness and 'usual day' flags
PAQ_J	Global Physical Activity Questionnaire: days and minutes of five activity domains

Table 3. NHANES files joined on the respondent id (SEQN).

A recall is kept only when NHANES marks it complete and the respondent called it a usual day of eating. Intake is the mean of the kept days. Weekly MET-minutes weight vigorous work and recreation at 8 METs and moderate activity and active travel at 4. Adults aged 18-79 who are not pregnant and whose intake is plausible (500-3,500 kcal for women, 800-4,200 kcal for men) remain: **3,811 adults**.

3.3 Recipe photos and original names

RAW_recipes.csv has no photos, and its names are lower-case with punctuation removed. A larger public scrape of Food.com (the parsed Food.com Recipes and Reviews dataset on Hugging Face) keeps photo URLs and original names but no id column. Every photo URL, however, contains the Food.com recipe id split into two-digit folders:

```
https://img.sndimg.com/food/image/upload/w_555,h_416,c_fit,.../v1/img/recipes/28/48/28/pic.jpg
recipe id 284828
```

Joining the folders back into an id gives an exact match to RAW_recipes. 43,247 of 78,930 recipes (54.8%) have a photo, rising to about 90% of recipes with five or more reviews, and the original names agree with the cleaned titles for about 95% of matched recipes. A first version of the parser read only the first folder, which silently matched 90 recipes; checking the join rate caught it.

3.4 Cleaning

Rule	Recipes failing
calories outside 20-2500 per serving	2,047
stated calories disagree with macros by more than 30%	3,130
cooking time 0 or over 24 hours	589

Table 4. Cleaning rules (a recipe can fail more than one). 78,930 of 83,782 kept.

Reviews are then restricted to surviving recipes: 221,732 reviews by 64,643 users. The user x recipe matrix is 0.0043% dense and half of all recipes have at most 2 review(s).

4. Environment Setup

4.1 Python environment

```
cd C:\Users\Gunasekaran\Dev\nutrition-recommender
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

4.2 A problem met during setup: Smart App Control

The first install pulled scikit-learn 1.9.1, and importing it failed with *DLL load failed while importing _sag_fast: An Application Control policy has blocked this file*. The Windows Code Integrity log showed Smart App Control blocking the newly downloaded compiled extension files. Disabling a security feature was not an acceptable fix. Instead, the working venv from an earlier project was checked, scikit-learn 1.9.0 was found to be trusted on this machine, and it was pinned in requirements.txt. FastAPI, pydantic-core and the Vite/Rollup binaries were tested for the same problem before any code depended on them.

4.3 Python packages

Package	Version	Used for
pandas	3.0.5	Tables: loading, joins, grouping
numpy	2.5.3	Arrays and linear algebra
scipy	1.18.1	Sparse matrices, optimisation (HiGHS), hierarchical clustering
scikit-learn	1.9.0	Regression, SVM, K-means, GMM, PCA, SVD, CART, ensembles, metrics
hmmlearn	0.3.3	Categorical Hidden Markov Model
pyarrow	25.0.1	Parquet caches and the photo scrape
matplotlib	3.11.1	Notebook figures
seaborn	0.13.2	Heatmaps and plot styling
jupyter	1.1.1	The lab notebook
fastapi	0.141.1	The NUTRIX API
uvicorn	0.52.4	Web server for the API
pydantic	2.13.5	Request validation
httpx	0.28.1	Test client for the API checks
reportlab	5.0.1	This report
gdown	6.2.0	Food.com download from Google Drive

Table 5. Python packages (versions read from the environment).

4.4 Frontend packages

Package	Version	Used for
react	19.3.0	User interface components
react-dom	19.3.0	Rendering to the browser
react-router-dom	7.18.3	Page navigation
recharts	3.10.1	Radar, bar and scatter charts
lucide-react	1.44.0	Icons
vite	8.3.0	Development server and production build
typescript	7.0.2	Type checking
@vitejs/plugin-react	6.1.1	JSX transform for Vite
Node.js	24.14.1	JavaScript runtime

Table 6. Frontend packages (versions read from node_modules).

4.5 Obtaining the data

```
.venv\Scripts\python.exe src\download_data.py
# RAW_recipes.csv, RAW_interactions.csv    Google Drive (gdown)
# DEMO_J, BMX_J, DR1TOT_J, DR2TOT_J, PAQ_J  https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2017/D
# foodcom_parsed_0000/0001.parquet         Hugging Face parquet conversion (photos and names)
```

4.6 Project structure

nutrition-recommender/	
data/raw/	Food.com CSV and parquet files, NHANES .xpt files
data/processed/	parquet caches and catalog.parquet (generated)
data/user/nutrix.db	profile, liked recipes, meal log (created by the API)
models/	bundle.joblib, metrics.json (generated by src/build.py)
notebooks/nutrition_ml_lab.ipynb	all lab programs, executed
src/ download_data.py	recipes.py energy.py photos.py course.py
archetypes.py	recommender.py planner.py build.py
api/main.py	FastAPI service
frontend/	React app: src/pages, src/components, api.ts, styles.css
tests/check_api.py	endpoint checks
tests/check_planner.py	meal planner checks
outputs/figures/	notebook figures
reports/build_report.py	builds this PDF
	outputs/app/ screenshots, check results

5. System Mechanism

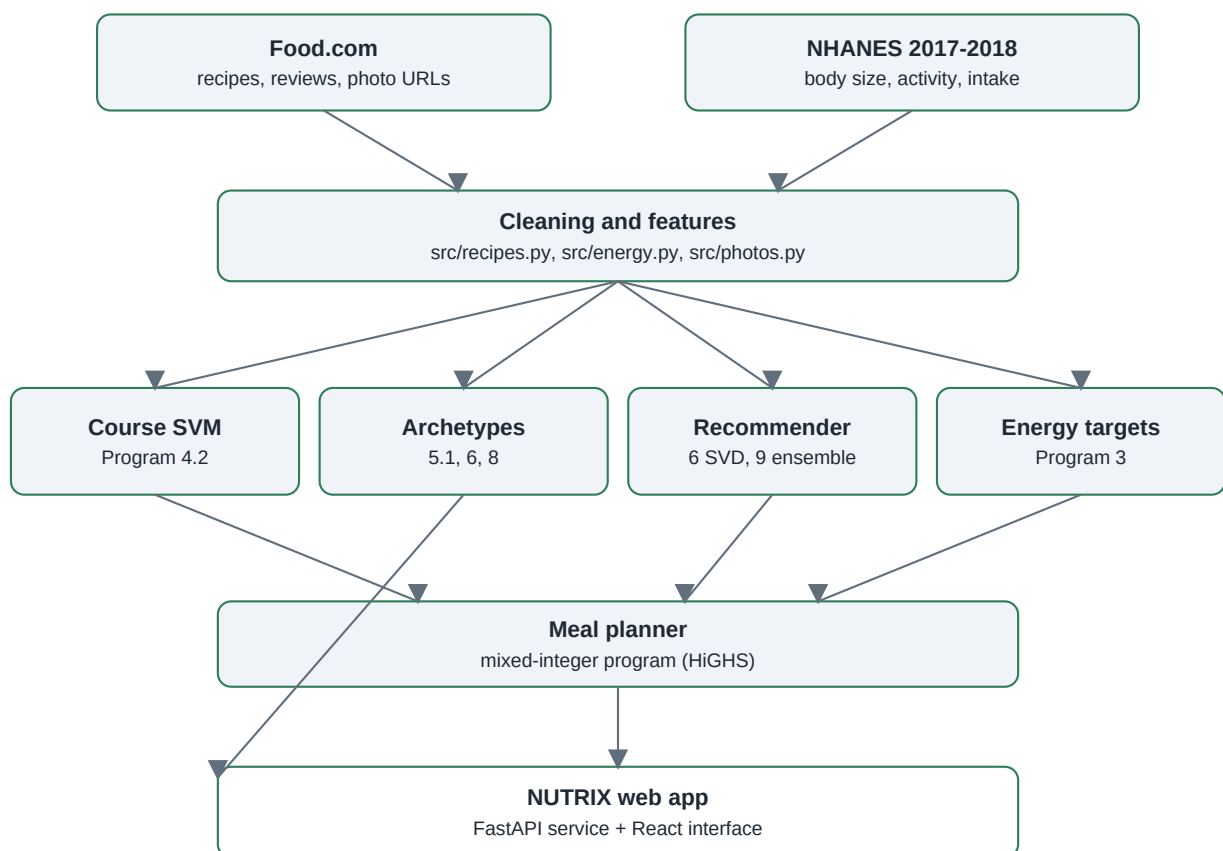


Figure 1. From raw data to the running application.

Stage	What happens	Where
1. Download	Food.com, NHANES and the photo scrape are saved to data/raw	src/download_data.py
2. Clean	Parse lists, convert nutrition to grams, apply cleaning rules, derive course and diet tags	src/recipes.py
3. Survey	Join NHANES files, apply recall rules, compute MET-minutes and formula targets	src/energy.py
4. Enrich	Fill missing courses with the SVM, assign archetypes and map coordinates, attach photos	src/course.py, archetypes.py, photos.py
5. Evaluate	Leave-last-out split, generators, candidate sets, ranker comparison	src/recommender.py via build.py
6. Train and save	Refit the recommender on all reviews, fit the intake regression, save the bundle and catalogue	src/build.py
7. Serve	Load models once; answer JSON requests; store the profile and meal log in SQLite	api/main.py
8. Interact	Render pages, charts and dialogs; call the API; show photos from the Food.com CDN	frontend/

Table 7. The eight stages.

6. The Code Layer

Function	What it does (src/recipes.py)
clean_recipes	Parses lists, converts %DV to grams, applies the three cleaning rules, derives course and diet flags
add_nutrient_features	Energy shares of protein, carbohydrate, fat, sugar and saturated fat; log sodium density; log calories
load_recipes / load_interactions	Cleaned tables with parquet caches
recipe_stats	Review counts and a mean rating shrunk toward the global mean by five pseudo-reviews

Function	What it does (src/energy.py)
mifflin_st_jeor, daily_target	Resting energy and the calorie and macronutrient targets
load_nhanes	The cleaned NHANES adults with intake, MET-minutes and formula estimates
intake_features, fit_intake_model	Regressors and the linear regression of reported intake

Function	What it does (src/course.py)
build_course_model	TF-IDF over ingredients plus scaled nutrition, into a calibrated linear SVM
fill_courses	Adds course_final, keeping tags and accepting model predictions with probability ≥ 0.5

Function	What it does (src/archetypes.py)
Archetypes	K-means with silhouette choice of k, centroid naming, CART rules and PCA map

Function	What it does (src/recommender.py)
temporal_split, Evaluation	Leave-last-out split and the shared evaluation setup
Popularity, Trending, PureSVD, ContentModel	The four candidate generators
HybridRecommender.candidates	Union of the generators' top 50 with the 23 ranking features
ranker_models, sample_negatives, pool_ranks	Program 9 models, training rows and ranking metrics
Function	What it does (src/planner.py)
plan_meals, _slot_options, _solve_day	Candidate selection and the per-day mixed-integer program
Function	What it does (src/photos.py)
load_photos, attach_photos	Photo URLs and original names joined through the id in each URL
Function	What it does (src/build.py)
main	Runs every step, compares rankers on validation users and writes the bundle, metrics and catalogue

Code, converting and cleaning the nutrition panel (src/recipes.py)

```
def clean_recipes(raw):
    df = raw.copy()
    df["tags"] = _parse_list(df["tags"])
    df["ingredients"] = _parse_list(df["ingredients"])
    df["steps"] = _parse_list(df["steps"])
    nutrition = pd.DataFrame(
        _parse_list(df["nutrition"]).tolist(), columns=NUTRITION_FIELDS, index=df.index
    )
    for col, dv in DAILY_VALUE.items():
        nutrition[col] = nutrition[col] / 100 * dv
    df = pd.concat([df.drop(columns=["nutrition"]), nutrition], axis=1)
    df = add_nutrient_features(df)
    ratio = df["atwater_kcal"] / df["calories"].where(df["calories"] > 0)
    keep = df["calories"].between(*KCAL_RANGE)
    keep &= ratio.between(*ATWATER_RANGE)
    keep &= df["minutes"].between(1, MAX_MINUTES)
    df = df.loc[keep].copy()
    df["title"] = df["name"].map(display_name)
    df["course"] = df["tags"].map(course_from_tags)
    for diet, tags in DIETS.items():
        df[diet] = df["tags"].map(lambda t, tags=tags: bool(tags & set(t)))
    df["submitted"] = pd.to_datetime(df["submitted"])
    df = df.rename(columns={"id": "recipe_id"})
    return df.reset_index(drop=True)
```

Code, the daily target (src/energy.py)

```
def daily_target(weight, height, age, male, activity="moderate", goal="maintain"):
    bmr = float(mifflin_st_jeor(weight, height, age, male))
    floor = 1500 if male else 1200
    kcal = max(bmr * PAL[activity] + GOAL_ADJUST[goal], floor)
    target = {"bmr": bmr, "kcal": kcal}
    for macro, share in MACRO_SPLIT[goal].items():
        target[macro] = kcal * share / KCAL_PER_GRAM[macro]
    return target
```

Part III — The Eleven Programs

Each chapter follows the same pattern: the aim, the theory needed to understand it, the steps, the code as it appears in the executed notebook, the result it printed, and how to read that result. Every random seed is 42.

7. Program 1 — Loading and Viewing the Datasets

Aim

Load the three datasets, confirm their shape, and look at what a row actually contains before modelling anything.

Theory

Loading is where silent errors enter: a list stored as text, a misread separator or a join on the wrong key all produce data that looks fine and models that mean nothing. Checking row counts, column types and a few rows against the source description is the cheapest test there is.

Steps

1. Read the raw Food.com CSVs and print their shapes.
2. Load the cleaned recipes, the reviews restricted to them and the NHANES adults.
3. Plot calories per serving, reviews per year and the distribution of stars.

Code

```
recipes = load_recipes()
interactions = load_interactions(recipes["recipe_id"])
nhanes = load_nhanes()
print(
    f"{len(recipes):,} recipes, {len(interactions):,} reviews by "
    f"{interactions['user_id'].nunique():,} users, {len(nhanes):,} NHANES adults"
)
recipes[["title", "course", "minutes", "n_ingredients", "calories", "protein_g",
        "carbs_g", "fat_g", "sugar_g", "sodium_mg"]].head()
```

Result obtained

```
raw recipes (83782, 12), raw reviews (731927, 5)
...
78,930 recipes, 221,732 reviews by 64,643 users, 3,811 NHANES adults
```

		title	course	minutes	n_ingredients	calories	protein_g	carbs
0	1	Brownies In The World Best Ever	dessert	40	9	138.4	1.5	18
1	1	In Canada Chocolate Chip Cookies	NaN	45	11	595.1	6.5	78
2		412 Broccoli Casserole	side	40	9	194.8	11.0	9
3		Millionaire Pound Cake	dessert	120	7	878.3	10.0	117
4		2000 Meatloaf	main	90	13	267.0	14.5	6

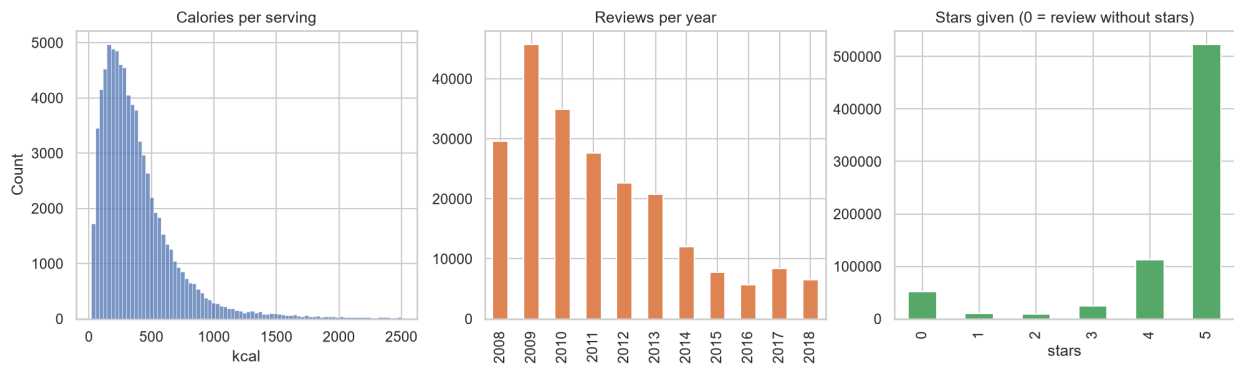


Figure 2. Calories per serving, reviews per year, and stars in the raw reviews.

How to read this

Calories per serving are right-skewed with most recipes between 150 and 500 kcal. Review activity peaked in 2009 and declined, which matters for evaluation: later users have less company. Almost all stars are 4 or 5, and the large bar at 0 is reviews without a star rating, not bad reviews. That is why reviews are used as implicit feedback.

8. Program 2 — Summary Statistics and Cleaning

Aim

Summarise the data statistically and use the summary to decide which rows can be trusted.

Theory

Food.com nutrition is computed from author-entered ingredients and serving counts, so it contains errors: whole recipes entered as one serving, zero-calorie rows, and panels whose calories disagree with their macronutrients. The Atwater identity gives a target-free consistency check, so cleaning on it cannot leak information into later models.

Steps

1. Convert the raw nutrition lists to grams.
2. Compare stated calories with $4P + 4C + 9F$.
3. Count rows failing each rule.
4. Describe the cleaned nutrients and the long tail of reviews.

Code

```
nutrition = pd.DataFrame(raw_recipes["nutrition"].map(ast.literal_eval).tolist(), columns=NUTRITIO
for col, dv in DAILY_VALUE.items():
    nutrition[col] = nutrition[col] / 100 * dv
atwater = 4 * nutrition["protein_g"] + 4 * nutrition["carbs_g"] + 9 * nutrition["fat_g"]
ratio = atwater / nutrition["calories"].where(nutrition["calories"] > 0)
rules = pd.Series({
    f"calories outside {KCAL_RANGE[0]}-{KCAL_RANGE[1]} per serving": (~nutrition["calories"].betwe
    "stated calories disagree with macros by more than 30%": (~ratio.between(*ATWATER_RANGE)).sum(
    "cooking time 0 or over 24 hours": (~raw_recipes["minutes"].between(1, MAX_MINUTES)).sum(),
}, name="rows failing (rules overlap)")
print(rules.to_string())
print(f"\nmedian Atwater / stated calories: {ratio.median():.3f}")
print(f"kept {len(recipes):,} of {len(raw_recipes):,} recipes ({len(recipes) / len(raw_recipes):.1
```

Result obtained

```
calories outside 20-2500 per serving          2047
stated calories disagree with macros by more than 30%  3130
cooking time 0 or over 24 hours                589
median Atwater / stated calories: 0.980
kept 78,930 of 83,782 recipes (94.2%)
```

```
half of recipes have at most 2 review(s); half of users wrote 1
review matrix density: 0.004346%
```

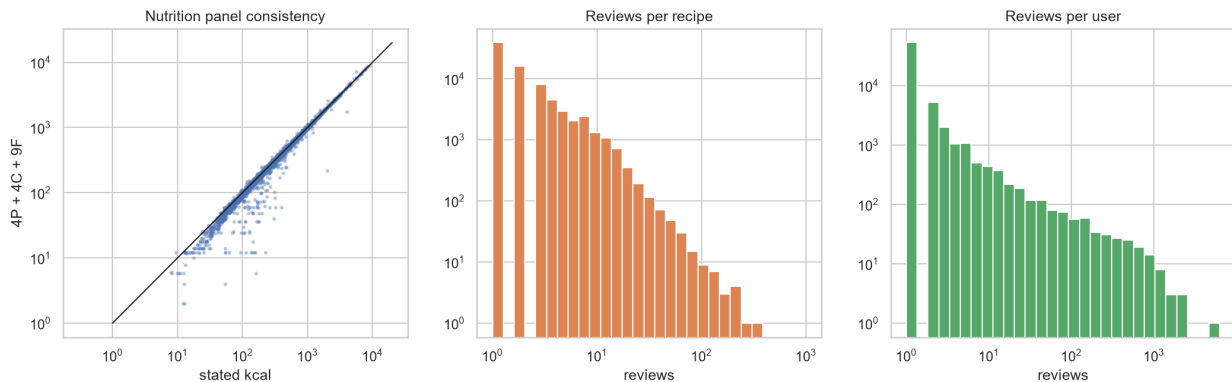


Figure 3. Nutrition panel consistency and the long tails of reviews per recipe and per user.

How to read this

The points on the diagonal are consistent panels; the clouds away from it are the rows removed. Both review histograms fall in a straight line on log-log axes: a few recipes and users account for most activity while most recipes have one or two reviews. This long tail limits every recommender result later in the report.

9. Program 3 — Linear Regression of Daily Energy Intake

Aim

Predict how much NHANES adults report eating from age, sex, body size and activity, and compare with the Mifflin-St Jeor target.

Theory

Ordinary least squares fits $y = b_0 + b_1 x_1 + \dots + b_k x_k$ by minimising the sum of squared residuals. R^2 is the share of variance explained, and can be negative for a model that is worse than predicting the mean, which is how an external formula is judged here. Activity enters as $\log(1 + \text{MET-minutes})$ because a few manual workers report tens of thousands of MET-minutes a week.

Steps

1. Build the regressors and an 80/20 split.
2. Fit the regression and print its coefficients.
3. Score it and the formula on the test set, and cross-validate the regression.

Code

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import KFold, cross_val_score, train_test_split
X, y = intake_features(nhanes), nhanes["intake"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
lr = LinearRegression().fit(X_train, y_train)
pred = lr.predict(X_test)
formula = nhanes.loc[X_test.index, "formula_tdee"]
print(pd.Series(lr.coef_, index=X.columns).round(2).to_string())
print(f"intercept {lr.intercept_:.1f}")
cv = cross_val_score(LinearRegression(), X, y, cv=KFold(5, shuffle=True, random_state=42), scoring='r2')
print(f"regression test R2 {r2_score(y_test, pred):.3f} RMSE {mean_squared_error(y_test, pred):.3f}")
print(f"formula test R2 {r2_score(y_test, formula):.3f} RMSE {mean_squared_error(y_test, formula):.3f}")
print(f"formula minus reported intake, mean: {(formula - y_test).mean():+.0f} kcal")
```

Result obtained

```
age          -2.52
male         309.55
weight        1.02
height       15.36
log_met       5.68
intercept -705.0
regression test R2 0.139 RMSE 682 kcal (5-fold CV R2 0.159)
formula test R2 -0.468 RMSE 891 kcal
formula minus reported intake, mean: +313 kcal
```

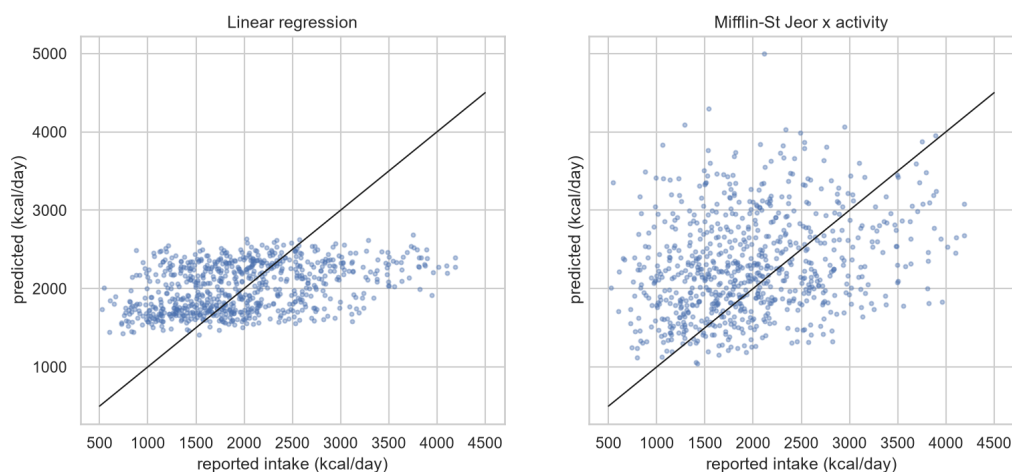


Figure 4. Predicted against reported intake.

How to read this

Being male adds about 310 kcal/day and each centimetre of height about 15 kcal/day. Still, the regression explains only 16% of the variance: people eat differently from day to day and a recall covers one or two days. The formula scores a negative R^2 because it sits +313 kcal above reported intake on average, the well-documented under-reporting of dietary recalls (Willett, 2012). The app keeps the formula as the target and shows the regression only as context.

10. Program 4.1 — Bayesian Logistic Regression

Aim

Estimate the probability that a recipe is a dessert from its nutrient profile alone, with uncertainty on every weight and every prediction.

Theory

Logistic regression models $P(y = 1 | x) = \text{sigmoid}(w \cdot x)$. A Bayesian treatment places a prior on w , here $N(0, 2^2)$ on each weight, and combines it with the likelihood. The posterior has no closed form, so the Laplace approximation (Bishop, 2006) finds its mode by optimisation and approximates it with a Gaussian whose covariance is the inverse Hessian of the negative log posterior at the mode. Predictions average the sigmoid over samples of w .

Steps

1. Sample 12,000 tagged recipes; standardise the seven nutrient features.
2. Minimise the negative log posterior with L-BFGS.
3. Compute the Hessian and its inverse.
4. Sample 2,000 weight vectors and form the posterior predictive.

Code

```
PRIOR_VAR = 2.0 ** 2
def neg_log_posterior(w):
    z = Xtr @ w
    return np.sum(np.logaddexp(0, z) - ytr * z) + w @ w / (2 * PRIOR_VAR)
def gradient(w):
    return Xtr.T @ (expit(Xtr @ w) - ytr) + w / PRIOR_VAR
w_map = minimize(neg_log_posterior, np.zeros(Xtr.shape[1]), jac=gradient, method="L-BFGS-B").x
p_tr = expit(Xtr @ w_map)
hessian = (Xtr * (p_tr * (1 - p_tr))[:, None]).T @ Xtr + np.eye(len(w_map)) / PRIOR_VAR
cov = np.linalg.inv(hessian)
sd = np.sqrt(np.diag(cov))
posterior = pd.DataFrame(
    {"mean": w_map, "sd": sd, "2.5%": w_map - 1.96 * sd, "97.5%": w_map + 1.96 * sd},
    index=["intercept"] + NUTRIENT_FEATURES,
).round(3)
posterior
```

Result obtained

	mean	sd	2.5%	97.5%
intercept	-2.654	0.065	-2.781	-2.527
protein_pct	-1.697	0.749	-3.165	-0.230
carbs_pct	0.684	1.388	-2.037	3.404
fat_pct	0.255	1.234	-2.163	2.674
sugar_pct	0.422	0.049	0.326	0.519
sat_fat_pct	0.746	0.042	0.664	0.829
log_sodium_density	-0.020	0.034	-0.087	0.046
log_kcal	0.080	0.033	0.014	0.145

test accuracy 0.828, ROC AUC 0.889, dessert share 0.180

	title	course	P(dessert)	95% interval	width
1550	Melon Fever	beverage	0.432	0.361 - 0.505	0.144
2606	Amaretto And Cream	beverage	0.639	0.567 - 0.708	0.141
2602	Swedish Dried Fruit Cream	condiment	0.277	0.211 - 0.352	0.141
2836	Vietnamese Pickled Carrots And Daikon Radish	snack	0.442	0.375 - 0.513	0.137
2897	Big Bob Gibson Barbecue Ribs	main	0.574	0.505 - 0.638	0.133
1461	California Fruit Cubes	beverage	0.482	0.415 - 0.547	0.132
1655	30 Second Steak Sauce	condiment	0.660	0.592 - 0.723	0.131
390	Emily S Creamy Pepperoni Dip	breakfast	0.468	0.403 - 0.534	0.131

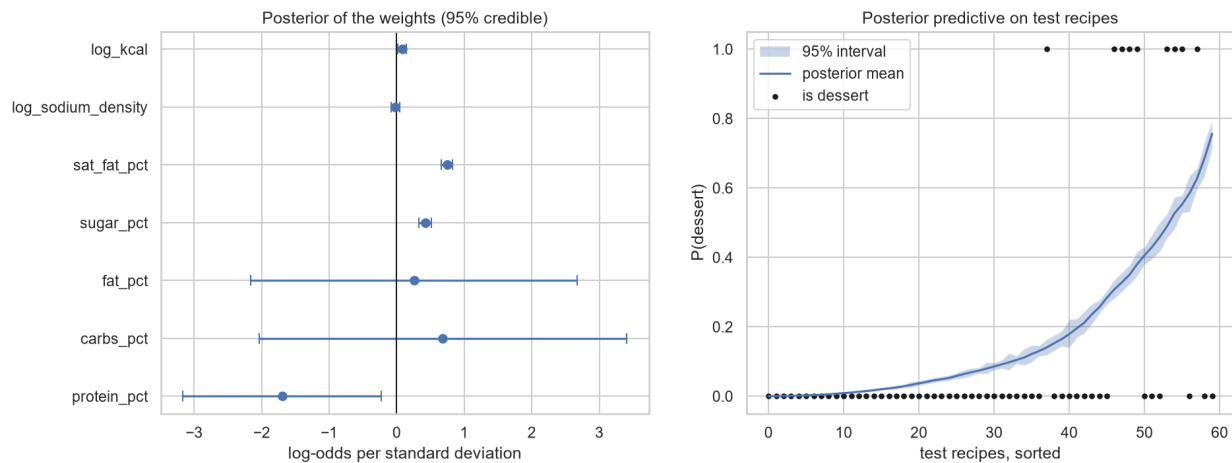


Figure 5. Credible intervals of the weights and the posterior predictive band.

How to read this

ROC AUC is 0.889. Accuracy (82.8%) is barely above always answering 'not a dessert' (82.0%), which is why AUC is the number to report. 4 of 7 weights have intervals excluding zero. The protein, carbohydrate and fat shares always sum to one, so the data cannot separate their effects, and the posterior shows exactly that as wide intervals where a point estimate would hide it. The widest predictive bands belong to sweet drinks, sauces and dips.

11. Program 4.2 — Support Vector Machine for the Course

Aim

Predict a recipe's course so the meal planner can place the one recipe in nine that has no course tag.

Theory

A linear SVM finds the hyperplane that separates classes with the largest margin, trading margin against misclassification through C. Multi-class problems are solved one class against the rest. SVM scores are not probabilities, so the model is wrapped in cross-validated calibration, which fits a mapping from scores to probabilities and lets the app accept only confident predictions. `class_weight='balanced'` stops the large main-dish class dominating. Tags are excluded from the features because the course tags are the labels.

Steps

1. Build TF-IDF weights over ingredient names and scale the numeric features.
2. Train a calibrated LinearSVC on 80% of tagged recipes.
3. Report per-class F1 and the confusion matrix.
4. Refit on all tagged recipes and fill untagged courses with probability ≥ 0.5 .

Code

```
def build_course_model(C=0.3, random_state=42):
    features = ColumnTransformer(
        [
            (
                "ingredients",
                TfidfVectorizer(analyzer=identity, min_df=3, sublinear_tf=True),
                "ingredients",
            ),
            ("nutrition", StandardScaler(), NUMERIC_FEATURES),
        ]
    )
    svm = LinearSVC(C=C, class_weight="balanced", random_state=random_state)
    return make_pipeline(features, CalibratedClassifierCV(svm, cv=3))
```

Result obtained

	precision	recall	f1-score	support
beverage	0.821	0.877	0.848	553
breakfast	0.680	0.459	0.548	1477
condiment	0.633	0.453	0.528	477
dessert	0.825	0.927	0.873	2535
main	0.784	0.861	0.821	5457
side	0.679	0.699	0.689	2711
snack	0.595	0.322	0.418	760
accuracy			0.756	13970
macro avg	0.717	0.657	0.675	13970
weighted avg	0.746	0.756	0.745	13970

```

course_source
tag      69850
model    7266
NaN      1814

                                title course_final course_confidence
...

```

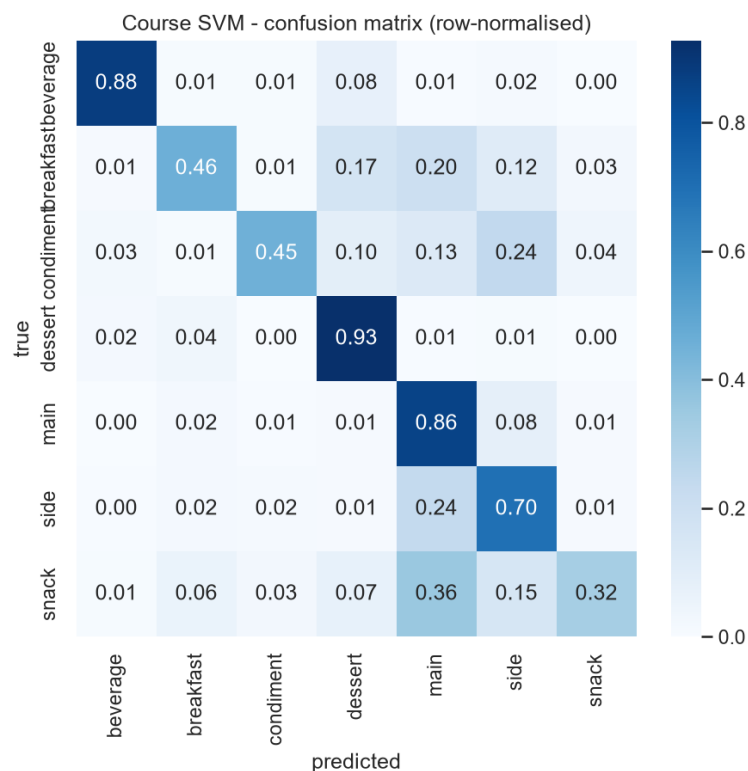


Figure 6. Row-normalised confusion matrix.

How to read this

Accuracy is 75.6% and macro F1 0.67. Desserts, beverages and mains are distinctive. Snacks are the hardest class: about a third are predicted as mains. Snack and breakfast are defined by when food is eaten more than by what is in it, so ingredients and nutrition cannot fully separate them. 7,266 untagged recipes gained a course and 1,814 stay out of the planner.

12. Program 5.1 — K-means Nutrition Archetypes

Aim

Group recipes by what they are like to eat, independent of their names.

Theory

K-means assigns each point to the nearest of k centroids and moves each centroid to the mean of its points, repeating until assignments stop changing; it minimises inertia, the total squared distance to centroids. Inertia always falls as k grows, so k is chosen by the silhouette score, which compares each point's distance to its own cluster with its distance to the nearest other cluster.

Steps

1. Standardise the seven portion-independent nutrient features.
2. Fit K-means for $k = 4$ to 10 and compute silhouette on a 5,000-recipe sample.
3. Fit the best k and name each centroid from fixed nutrition thresholds.

Code

```
from sklearn.cluster import KMeans
archetypes = Archetypes().fit(recipes)
recipes["archetype"] = archetypes.label(recipes)
Z = archetypes.scaler_.transform(recipes[NUTRIENT_FEATURES])
inertia = {k: KMeans(k, n_init=10, random_state=42).fit(Z).inertia_ for k in archetypes.k_range}
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(list(inertia), list(inertia.values()), marker="o")
axes[0].set(title="Elbow", xlabel="k", ylabel="inertia")
axes[1].plot(list(archetypes.silhouette_), list(archetypes.silhouette_.values()), marker="o", color="orange")
axes[1].axvline(archetypes.k_, color="k", ls="--")
axes[1].set(title=f"Silhouette (chosen k = {archetypes.k_})", xlabel="k", ylabel="silhouette")
save(fig, "p5_kmeans_selection")
...
```

Result obtained

	archetype	share	kcal	protein %	carbs %	fat %	sugar %	sat fat %	sodium mg/100kc
0	Sugary & Light	0.076	139.0	10.0	75.0	14.0	48.0	5.0	138
1	Rich & Hearty	0.220	572.0	21.0	22.0	57.0	5.0	21.0	131
2	Starchy	0.193	337.0	17.0	59.0	25.0	8.0	7.0	136
3	Sugary & Unsalted	0.037	148.0	3.0	94.0	3.0	80.0	1.0	1
4	High-protein	0.097	275.0	47.0	18.0	35.0	7.0	10.0	168
5	Buttery	0.085	262.0	12.0	15.0	73.0	7.0	41.0	84
6	Unsalted & Light	0.038	162.0	8.0	45.0	47.0	16.0	11.0	0
7	Rich & Light	0.110	144.0	12.0	30.0	59.0	11.0	15.0	159
8	Sweet	0.145	357.0	6.0	52.0	42.0	33.0	21.0	53

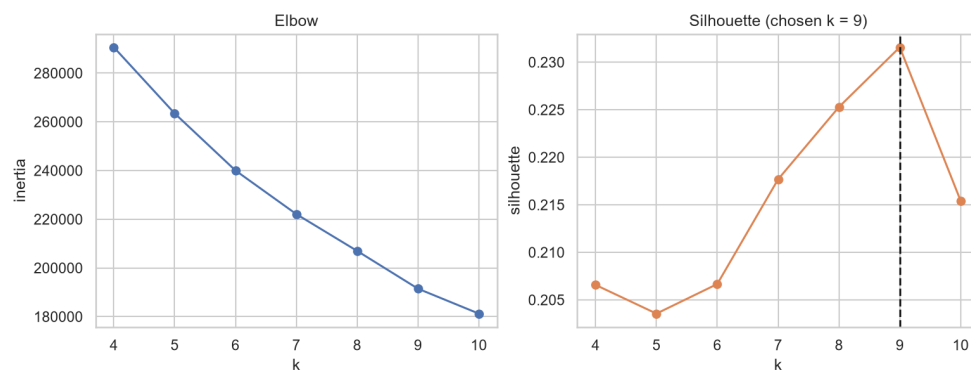


Figure 7. Elbow and silhouette.

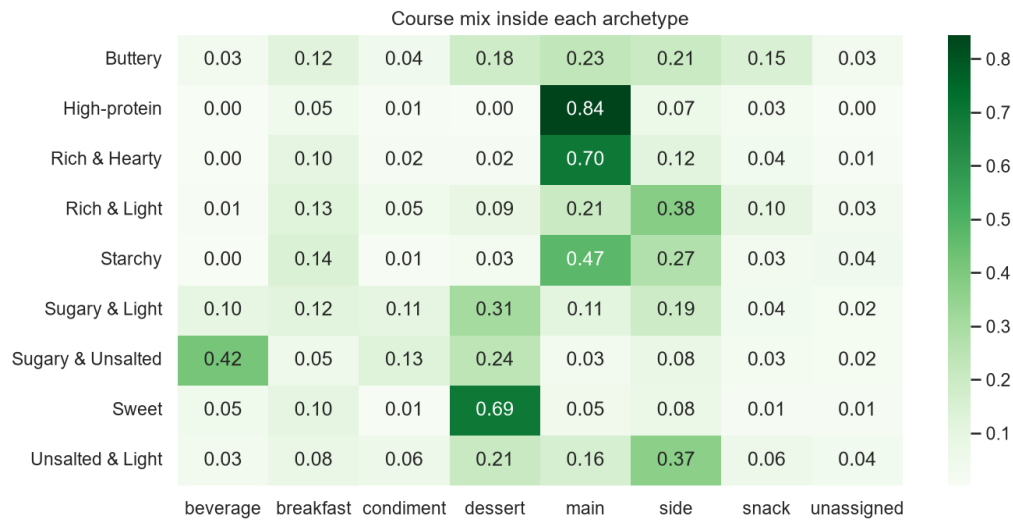


Figure 8. Course mix inside each archetype.

How to read this

The silhouette peaks at $k = 9$ but stays modest, which is honest: nutrition is a continuum. The archetypes are still useful because their course mixes differ sharply, for example sugary archetypes are mostly desserts and beverages while rich hearty ones are mostly mains.

13. Program 5.2 — Gaussian Mixture Model of User Taste**Aim**

Find taste segments among users and measure how cleanly users belong to them.

Theory

A Gaussian mixture models data as a weighted sum of Gaussian components and is fitted by Expectation Maximisation, alternating soft assignment (responsibilities) and parameter updates. BIC balances fit against the number of parameters. Proportions near zero break the Gaussian assumption, so shares are square-root transformed (the Hellinger transform), and a covariance floor stops components collapsing onto users who never cook an archetype.

Steps

1. Describe each user with 10+ reviews by the share of reviews in each archetype.
2. Square-root transform.
3. Fit full-covariance mixtures for 1 to 10 components.
4. Choose the simplest model within 2% of the best BIC.

Code

```
from sklearn.mixture import GaussianMixture
labels = pd.Series(archetypes.predict(recipes), index=recipes["recipe_id"])
inter = interactions.assign(archetype=labels.reindex(interactions["recipe_id"]).to_numpy())
per_user = inter.groupby("user_id").size()
active = inter[inter["user_id"].isin(per_user[per_user >= 10].index)]
taste = pd.crosstab(active["user_id"], active["archetype"], normalize="index")
taste.columns = [archetypes.names[c] for c in taste.columns]
X_taste = np.sqrt(taste)
def fit_gmm(k):
    return GaussianMixture(k, covariance_type="full", reg_covar=1e-3, n_init=5, random_state=42).fit(X_taste)
bic = {k: fit_gmm(k).bic(X_taste) for k in range(1, 11)}
min_bic_k = min(bic, key=bic.get)
# BIC keeps creeping down as components are added, so take the simplest
# model whose BIC is within 2% of the best.
best_k = min(k for k, v in bic.items() if v <= bic[min_bic_k] + 0.02 * abs(bic[min_bic_k]))
gmm = fit_gmm(best_k)
...
```

Result obtained

1,680 users; lowest BIC at 9 components, simplest within 2%: 5
users with a segment probability above 0.8: 95.1%

	Sugary & Light	Rich & Hearty	Starchy	Sugary & Unsalted	High-protein	Bu
segment 1 (182 users)	0.4	-5.3	-3.4	0.8	5.0	
segment 2 (229 users)	-1.2	6.6	4.6	-1.3	3.0	
segment 3 (227 users)	0.6	-1.9	1.3	0.7	-10.7	
segment 4 (437 users)	-0.5	2.4	-1.2	-0.9	2.8	
segment 5 (605 users)	0.5	-2.0	-0.3	0.7	-0.6	

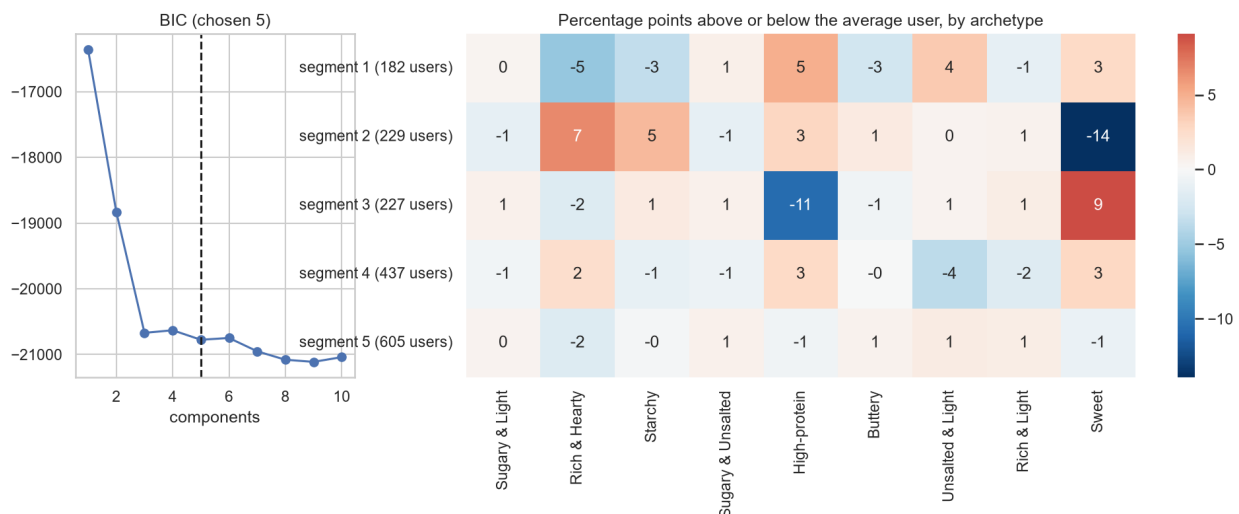


Figure 9. BIC, and each segment's difference from the average user in percentage points.

How to read this

95.1% of users belong to one of 5 segments with probability above 0.8. 2 segments show a clear leaning of 8 or more points, such as a savoury group cooking far fewer sweet recipes and a baking group cooking more; the largest segment sits on the average. Before the transform and floor, BIC kept improving up to the largest model tried, a sign of degenerate components rather than real structure.

14. Program 5.3 — Hierarchical Clustering of Ingredients

Aim

Discover which ingredients are cooked together, without labels.

Theory

Agglomerative clustering starts with every item alone and repeatedly merges the closest pair of clusters. The distance between two ingredients is the Jaccard distance between the sets of recipes containing them, and average linkage uses the mean distance between members. The dendrogram records every merge and its height.

Steps

1. Take the 60 most common ingredients after salt, pepper and water.
2. Build a recipe x ingredient presence matrix.
3. Compute Jaccard distances and average linkage; draw the dendrogram.

Code

```
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import pdist
from sklearn.preprocessing import MultiLabelBinarizer
counts = recipes["ingredients"].explode().value_counts()
top = counts.drop(labels=["salt", "water", "pepper", "salt and pepper", "black pepper"], errors="i")
occurs = MultiLabelBinarizer(classes=list(top)).fit_transform(recipes["ingredients"]).astype(bool)
tree = linkage(pdist(occurs.T, metric="jaccard"), method="average")
fig, ax = plt.subplots(figsize=(9, 14))
dendrogram(tree, labels=list(top), orientation="left", leaf_font_size=9, ax=ax, color_threshold=0.
ax.set(title="Ingredients cooked together (Jaccard distance, average linkage)", xlabel="distance")
...
```

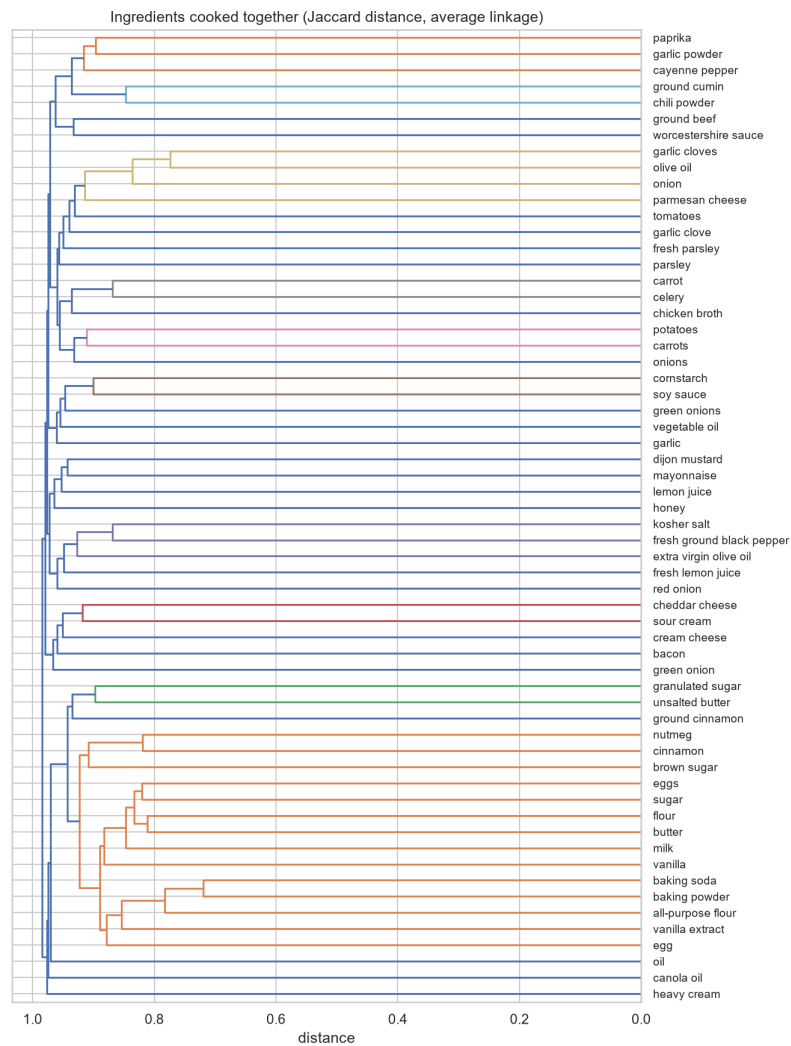


Figure 10. Ingredients cooked together.

How to read this

The tightest group is baking (flour, sugar, eggs, butter, milk, vanilla, baking powder and soda, cinnamon, nutmeg). Smaller groups form an Italian-style base (garlic, olive oil, onion, parmesan, tomatoes), a spice rub (paprika, garlic powder, cayenne, cumin, chili powder, ground beef), a stir-fry (soy sauce, cornstarch, green onions), a soup base (carrot, celery, chicken broth) and a cheese-and-bacon group. Distances above 0.7 are normal: even common ingredients share only part of their recipes.

15. Program 6 — PCA and Truncated SVD

Aim

Compress nutrition into a two-dimensional recipe map, and find the latent factors behind the review matrix.

Theory

PCA finds orthogonal directions of maximum variance; each component's loadings say how much each feature contributes. Truncated SVD factorises a matrix as $U S V^T$ keeping the top k singular values; applied to the sparse user $\times$ recipe matrix it gives the latent factors used by PureSVD (Cremonesi et al., 2010), which scores a user's recipes as $h V^T V$ for their history vector h .

Code

```
from sklearn.decomposition import PCA, TruncatedSVD
pca = PCA().fit(Z)
loadings = pd.DataFrame(pca.components_[1:3].T, index=NUTRIENT_FEATURES, columns=["PC1", "PC2", "PC3"])
xy = archetypes.coordinates(recipes)
recipes["map_x"], recipes["map_y"] = xy[:, 0], xy[:, 1]
fig, axes = plt.subplots(1, 3, figsize=(19, 5), gridspec_kw={"width_ratios": [1, 1, 1.6]})
...
```

```
class PureSVD:
    def __init__(self, n_factors=64, random_state=42):
        self.n_factors = n_factors
        self.random_state = random_state
    def fit(self, X):
        svd = TruncatedSVD(self.n_factors, n_iter=7, random_state=self.random_state)
        svd.fit(X)
        self.components_ = svd.components_.astype(np.float32)
        self.explained_variance_ratio_ = svd.explained_variance_ratio_
        return self
    def user_factors(self, H):
        return np.asarray(H @ self.components_.T, dtype=np.float32)
    def score(self, H):
        return self.user_factors(H) @ self.components_
```

Result obtained

first two components explain 65.5% of the variance
matrix 64,643 users x 78,930 recipes, 221,732 non-zeros
64 factors explain 26.8%, 128 explain 36.5%

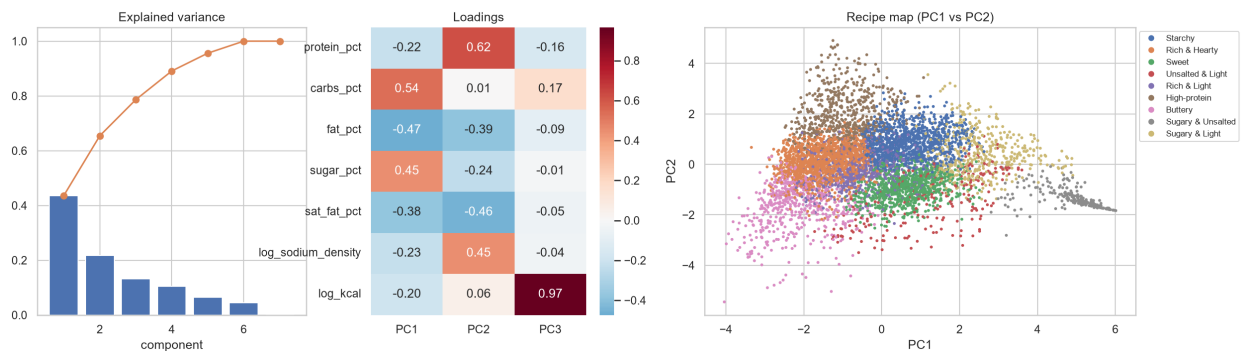


Figure 11. Explained variance, loadings and the recipe map.

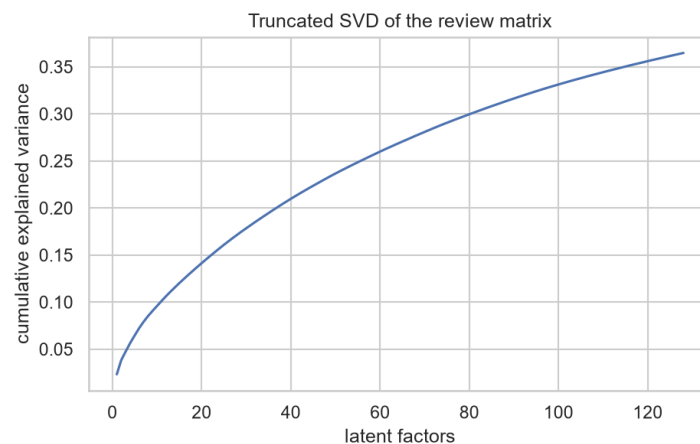


Figure 12. Cumulative variance of the truncated SVD.

How to read this

Two components explain 65.5%. PC1 contrasts carbohydrate (+0.54) and sugar (+0.45) against fat (-0.47); PC2 separates lean, salty, protein-rich dishes (protein +0.62) from fatty ones; PC3 is portion size (+0.97). 64 SVD factors explain only 26.8% of the review matrix, a direct view of how little users' histories overlap.

16. Program 7 — Hidden Markov Model of Eating Phases

Aim

Test whether users cook in phases, runs of similar recipes, rather than choosing each recipe independently.

Theory

A Hidden Markov Model has hidden states that follow a Markov chain (the transition matrix) and emit observations with state-specific probabilities (the emission matrix). Baum-Welch, an EM algorithm, fits both from unlabelled sequences (Rabiner, 1989). Here the observation is the archetype of each reviewed recipe, in date order.

Code

```
from hmmlearn.hmm import CategoricalHMM
long_users = per_user[per_user >= 20].index
seq = inter[inter["user_id"].isin(long_users)].sort_values(["user_id", "date"], kind="stable")
lengths = seq.groupby("user_id", sort=True).size().to_numpy()
obs = seq["archetype"].to_numpy().reshape(-1, 1)
fits = {}
for n in range(2, 6):
    model = CategoricalHMM(n_components=n, n_features=archetypes.k_, n_iter=200, random_state=42)
    model.fit(obs, lengths)
    fits[n] = model
    print(f"{n} states: log-likelihood {model.score(obs, lengths):.0f}, BIC {model.bic(obs, lengths):.0f}")
n_states = min(fits, key=lambda n: fits[n].bic(obs, lengths))
hmm = fits[n_states]
print(f"\n{n_states} users, {len(obs):,} reviews, BIC chooses {n_states} states")
```

Result obtained

```
2 states: log-likelihood -261,241, BIC 522,705
3 states: log-likelihood -258,702, BIC 517,779
4 states: log-likelihood -258,064, BIC 516,679
5 states: log-likelihood -258,636, BIC 518,024
884 users, 126,304 reviews, BIC chooses 4 states
expected reviews before switching state: state 1: 1.8, state 2: 442.3, state 3: 5.4, state 4: 18.7
```

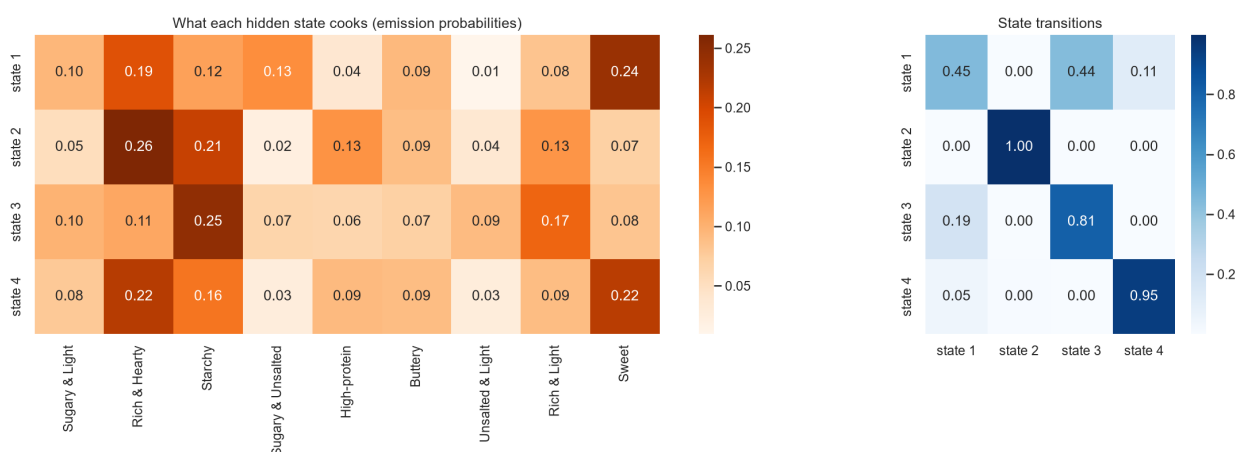


Figure 13. Emission and transition probabilities.

How to read this

BIC chose 4 states and 3 of them persist, with self-transition probability above one half: users do cook in runs. One state is effectively absorbing, so it describes a stable taste type rather than a passing phase. This supports weighting recent history when recommending.

17. Program 8 — CART Rules for the Archetypes**Aim**

Turn the K-means archetypes into threshold rules a person can read.

Theory

CART grows a binary tree by choosing, at each node, the feature and threshold that most reduce Gini impurity. Fitted to cluster labels it acts as a surrogate model, and fidelity, the share of recipes where tree and K-means agree, measures how faithful the rules are.

Code

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
Xn = recipes[NUTRIENT_FEATURES].to_numpy()
km_labels = archetypes.predict(recipes)
depth_fidelity = {
    d: (DecisionTreeClassifier(max_depth=d, min_samples_leaf=200, random_state=42).fit(Xn, km_labels)
        for d in range(1, 9))
}
print(pd.Series(depth_fidelity, name="fidelity").round(3).to_string())
print(f"\nthe app uses depth 4: fidelity {archetypes.tree_fidelity_:.1%}\n")
...
```

Result obtained

```
1    0.392
2    0.563
3    0.694
4    0.800
5    0.825
6    0.849
7    0.873
8    0.883
the app uses depth 4: fidelity 80.0%
|--- carbs_pct <= 0.41
|   |--- protein_pct <= 0.35
|   |   |--- log_kcal <= 5.70
|   |   |   |--- sat_fat_pct <= 0.30
|   |   |   |   |--- class: Rich & Light
|   |   |   |   |--- sat_fat_pct > 0.30
|   |   |   |   |--- class: Buttery
|   |   |--- log_kcal > 5.70
|   |   |   |--- sat_fat_pct <= 0.34
|   |   |   |   |--- class: Rich & Hearty
|   |   |   |   |--- sat_fat_pct > 0.34
|   |   |   |   |--- class: Buttery
|   |--- protein_pct > 0.35
|   |   |--- fat_pct <= 0.54
|   |   |   |--- log_kcal <= 6.47
...
```

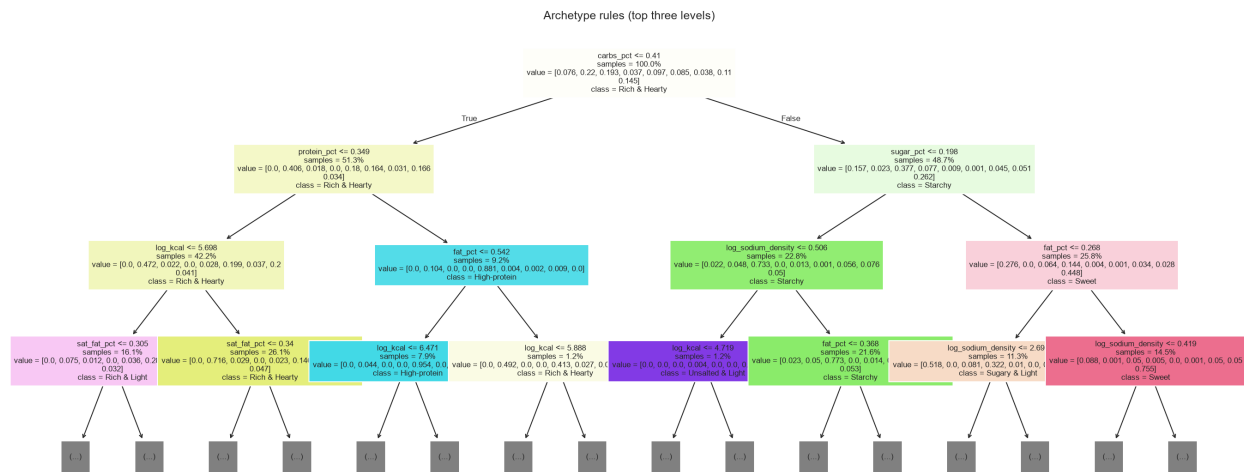


Figure 14. Top three levels of the archetype rules.

How to read this

Depth 4, used in the app, reproduces 80.0% of the labels. Each extra level adds only one to three points (depth 8 reaches 88%) while doubling the rules, so depth 4 is the readable compromise.

18. Program 9 — Ensemble Learning for Recommendation

Aim

Rank each user's candidate recipes so the recipe they cook next comes first, and compare ensemble methods.

Theory

Bagging (random forest) averages trees trained on bootstrap samples with random feature subsets, reducing variance. Boosting (histogram gradient boosting) fits trees sequentially to the errors of earlier trees, reducing bias. Soft voting averages the probabilities of several models. A linear logistic regression is the baseline.

Steps

1. Split each user's history in time: last recipe is the test target, the one before the validation target (users with 3+ recipes).
2. Fit the four generators on what remains; serve each target on the day it was reviewed and hide recipes not yet submitted.
3. Build candidates from each generator's top 50 with 23 features per user-recipe pair.
4. Train rankers on validation targets with 30 sampled negatives per user; choose on held-out validation users.
5. Score every model on the test targets.

Code, the candidate generators

```
class Trending:
    def __init__(self, window_months=12):
        self.window_months = window_months
    def fit(self, interactions, index):
        month = interactions["date"].to_numpy().astype("datetime64[M]").astype(np.int64)
        self.first_month_ = int(month.min())
        rows = month - self.first_month_
        counts = sp.coo_matrix(
            (np.ones(len(rows), np.float32), (rows, index.positions(interactions["recipe_id"]))),
            shape=(int(rows.max()) + 1, len(index)),
        ).toarray()
        self.cumulative_ = np.vstack(
            [np.zeros((1, len(index)), np.float32), counts.cumsum(axis=0, dtype=np.float32)]
        )
        return self
    def score(self, ref_day):
        month = np.asarray(ref_day).astype("datetime64[D]").astype("datetime64[M]").astype(np.int64)
        hi = np.clip(month - self.first_month_, 0, len(self.cumulative_) - 1)
        lo = np.clip(hi - self.window_months, 0, None)
        ...
```

Code, training and scoring the rankers

```
val = ev.candidate_frame(rec_eval, "val")
test = ev.candidate_frame(rec_eval, "test")
train_rows = sample_negatives(val)
print(f"candidates per user: {len(test) / len(ev.users):.0f}; true next recipe retrieved for {test}")
print(f"ranker training rows: {len(train_rows):,} ({train_rows['label'].sum()} positives)")
results, fitted = {}, {}
for name, model in ranker_models().items():
    model.fit(train_rows[FEATURES], train_rows["label"])
    results[name] = ranking_metrics(pool_ranks(model.predict_proba(test[FEATURES])[:, 1], test, le
    fitted[name] = model
rankers = pd.DataFrame(results).T
comparison = pd.concat([generators.assign(stage="generator"), rankers.assign(stage="ranker")])
comparison.round(4)
```

Result obtained

```
6,453 evaluation users
test targets never reviewed in training: 21.3%
      HR@10  NDCG@10  MRR
svd      0.005269  0.003202  0.003326
content  0.010693  0.006943  0.006860
popularity 0.013482  0.006048  0.005638
trending  0.007593  0.003666  0.003799
```

```
candidates per user: 181; true next recipe retrieved for 8.1% of users
ranker training rows: 15,035 (485 positives)
```

	HR@10	NDCG@10	MRR	stage
svd	0.0053	0.0032	0.0033	generator
content	0.0107	0.0069	0.0069	generator
popularity	0.0135	0.0060	0.0056	generator
trending	0.0076	0.0037	0.0038	generator
logistic regression	0.0201	0.0119	0.0112	ranker
random forest	0.0373	0.0224	0.0194	ranker
gradient boosting	0.0369	0.0217	0.0186	ranker
soft voting	0.0361	0.0221	0.0194	ranker

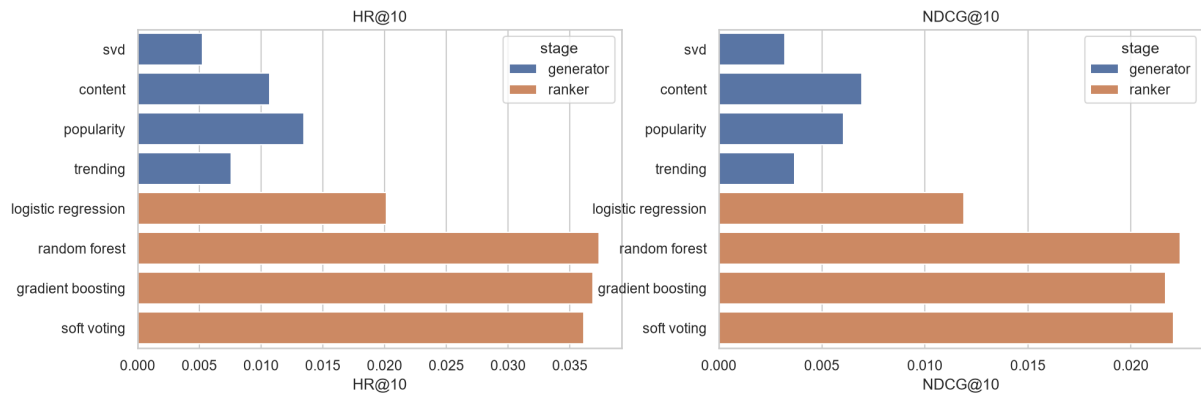


Figure 15. HR@10 and NDCG@10 for every generator and ranker.

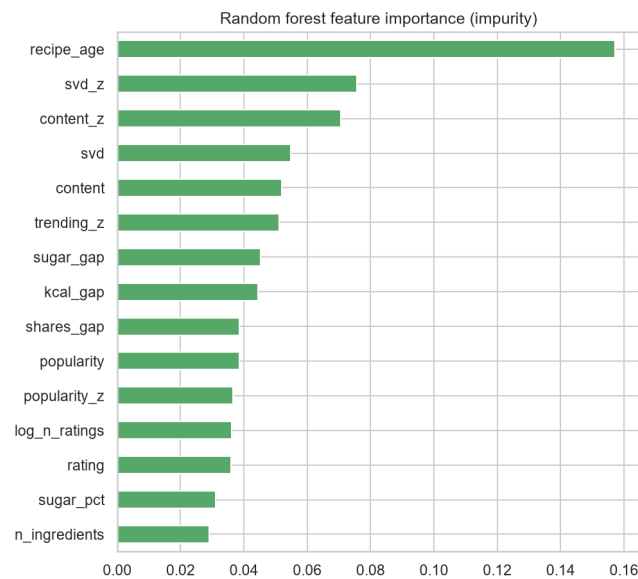


Figure 16. Random forest feature importance.

How to read this

The random forest ranker, chosen on validation users, finds the next recipe for 3.73% of users: 2.8 times the best generator (popularity, 1.35%) and about 295 times random. All tree ensembles perform alike and clearly beat logistic regression, so the gain comes from non-linear interactions between signals. Recipe age is the most important feature. The absolute numbers are limited by the data: 21.3% of test recipes had no training reviews, and retrieval finds the true recipe for only 8.1% of users.

An early version of the evaluation used each user's last training date as the serving day. Recipes submitted after that day could then be recommended, which quietly leaks the fact that a recipe will exist. The final evaluation serves each target on its own review date and hides recipes that did not yet exist.

Part IV — Application, Evaluation and Honest Limits

19. The Meal Planner

Aim

Turn recommendations into day or week plans that meet calorie, macronutrient, sugar and sodium targets.

Formulation

For each day a binary variable $x(s, r, q)$ selects recipe r at portion q (0.5, 1, 1.5 or 2 servings) for slot s (breakfast 25% of energy, lunch 35%, dinner 30%, snack 10%). The program minimises:

- the relative gap of the day's calories (weighted 3), protein (1.5), carbohydrate and fat (1 each) from target;
- sugar above 10% of energy (WHO, 2015) and sodium above 2,300 mg;
- a small penalty for portions other than one serving;
- a penalty of $0.3 \times (1 - \text{preference})$ per chosen recipe.

subject to exactly one recipe and portion per slot and no recipe twice in a day. Days are solved in sequence with used recipes removed, so a week never repeats a dish.

Code, choosing each slot's candidates

```
def _slot_options(recipes, preference, slot, target, n, course_col, rng, exploration):
    spec = SLOTS[slot]
    lo, hi = PORTION_KCAL_RANGE
    slot_kcal = spec["share"] * target["kcal"]
    eligible = recipes[
        recipes[course_col].isin(spec["courses"])
        & recipes["calories"].between(lo * slot_kcal / max(PORTIONS), hi * slot_kcal / min(PORTION
    ]
    if eligible.empty:
        raise ValueError(f"No recipes fit the {slot} slot with the current filters.")
    ranking = preference.reindex(eligible.index).fillna(0).to_numpy()
    if exploration:
        ranking = ranking + exploration * rng.gumbel(size=len(ranking))
    goal = np.array([4 * target["protein"], 4 * target["carbs"], 9 * target["fat"]]) / target["kca
    balance = np.abs(eligible[["protein_pct", "carbs_pct", "fat_pct"]].to_numpy() - goal).sum(axis
    if np.ptp(ranking) == 0:
        # No preference to rank by; every candidate is chosen for balance.
        top = eligible.index[np.argsort(balance)[: 2 * n]]
    else:
        by_preference = eligible.index[np.argsort(-ranking)[:n]]
        by_balance = eligible.index[np.argsort(balance - 0.3 * ranking)[:n]]
        top = by_preference.union(by_balance)
    options = []
    for rid in top:
        for p in PORTIONS:
            if lo * slot_kcal <= recipes.at[rid, "calories"] * p <= hi * slot_kcal:
                options.append((slot, rid, p))
    return options
```

Design decisions from experiments

- **Balanced candidates.** With only the most preferred recipes, a person who likes curries got a week 47% short on carbohydrate. Each slot now takes the 15 most preferred plus the 15 closest to the target's macro balance.
- **Day by day.** Solving a week as one program found no feasible plan in 20 seconds.
- **No per-slot calorie penalty.** It made the program so hard that plans stopped by the time limit missed calories by 21%. The portion band keeps slots sensible without it.

- **One-second limit with a guard.** Proving optimality takes several seconds per day. The solver stops after one second, and a day more than 3% off its calorie target is solved again with three times the candidates and five times the limit.

Program	Profile	Limit per day	Week time	Largest calorie gap	Largest macro gap
with slot term, 30 candidates	vegetarian woman	30 s (optimal)	43 s	0.4%	12.8%
with slot term, 30 candidates	vegetarian woman	1 s	8 s	21.1%	33.0%
with slot term, 30 candidates	man who likes curries	1 s	9 s	2.8%	26.1%
final: no slot term, 15 + 15	vegetarian woman	1 s	8 s	0.6%	9.9%
final: no slot term, 15 + 15	man who likes curries	1 s	8 s	2.2%	5.1%

Table 8. The experiment behind the planner design (single runs).

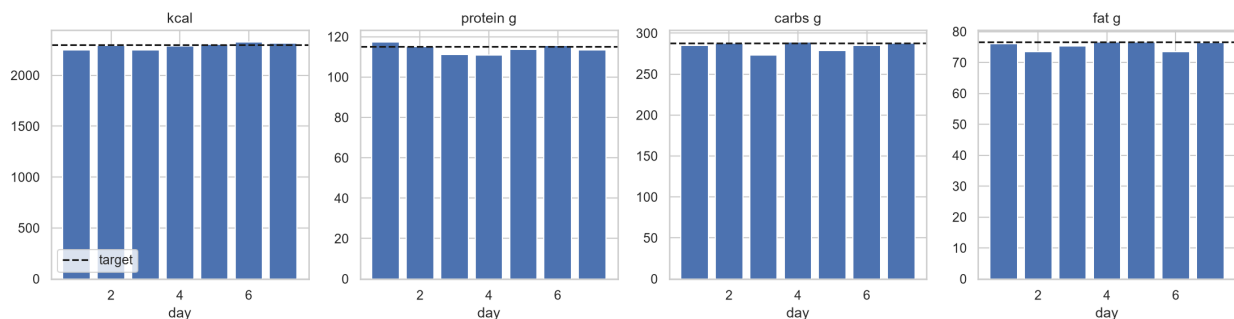


Figure 17. Seven-day plan for the notebook's example profile; every total within 5.1% of target.

20. The NUTRIX Application

20.1 Is it localhost or an application?

Both, in the precise sense: NUTRIX is a web application that runs locally. When you start it, a small web server (Uvicorn running the FastAPI service) starts on your own computer, and you use the application in a web browser at **<http://localhost:8000>**. *localhost* simply means 'this computer'. Nothing is uploaded, there is no login, and the profile and meal log stay in a file on your machine. The only outside connection is the browser loading recipe photos from Food.com's image server.

Question	Answer
Do I install it like a mobile or desktop app?	No. You start the server with one command and open the browser.
Can other people use it over the internet?	Not as built. It listens only on this computer. It can be deployed (Chapter 24).
Does it need the internet?	Only for recipe photos; everything else works offline.
Why a web application at all?	One codebase gives a rich interface on any computer or phone browser, and it is the standard way ML models are served.

20.2 Architecture

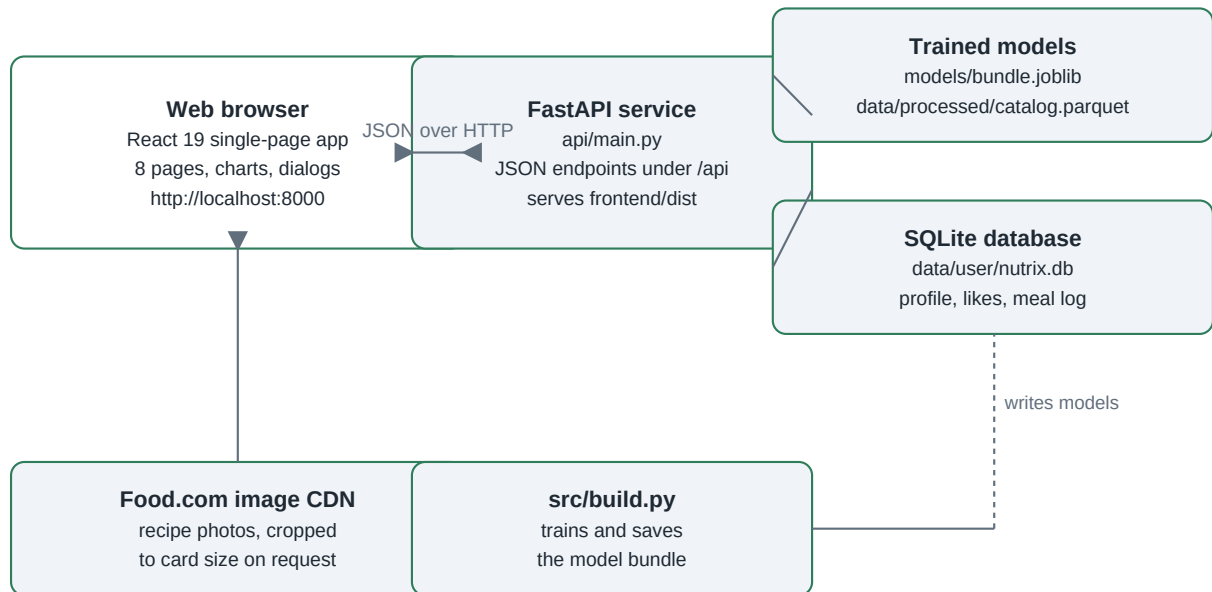


Figure 18. NUTRIX architecture.

The React app is compiled by Vite into static files in `frontend/dist`. FastAPI serves those files and the JSON API from one process. At start-up the API loads the trained model bundle and the recipe catalogue into memory once, so each request only runs inference. The meal log, profile and liked recipes are stored in SQLite, which needs no separate database server.

20.3 Technology stack

Layer	Technology	Why
Interface	React 19.3.0, React Router 7.18.3, TypeScript	Component-based pages with type-checked API data
Charts and icons	Recharts 3.10.1, lucide-react 1.44.0	Radar, bar and scatter charts; consistent icons
Build tool	Vite 8.3.0	Fast development server with /api proxy; optimised production build
API	FastAPI 0.141.1, pydantic 2.13.5, Uvicorn 0.52.4	Typed endpoints with automatic request validation
Storage	SQLite (Python standard library)	Single-file local database, no server
Models	scikit-learn 1.9.0, SciPy 1.18.1 (HiGHS)	Everything trained by <code>src/build.py</code>

Table 9. Technology stack.

20.4 API endpoints

Method	Path	Purpose
GET	/api/meta	Options for forms: activity levels, goals, diets, courses, archetypes, photo count
GET	/api/profile	Profile, targets, chip labels and the NHANES intake estimate
PUT	/api/profile	Partial profile update, validated by pydantic
GET	/api/recommendations	Ranked recipes with match score, reasons and the liked recipe each resembles
GET	/api/recipes	Food Library search by name or ingredient, with filters, sort and paging
GET	/api/recipes/{recipe_id}	Recipe detail: photo, ingredients, steps, nutrition, similar recipes
GET	/api/likes	Liked recipes
POST	/api/likes/{recipe_id}	Like a recipe
DELETE	/api/likes/{recipe_id}	Unlike a recipe
GET	/api/log	One day's planned and eaten meals with totals
POST	/api/log	Add a meal to a day and slot as planned or eaten
PATCH	/api/log/{entry_id}	Change a meal's status, servings or slot
DELETE	/api/log/{entry_id}	Remove a meal
DELETE	/api/log	Clear the whole log
POST	/api/plan	Generate a 1-7 day plan with the optimiser
POST	/api/plan/save	Save a generated plan as planned meals
GET	/api/progress	Daily totals and on-target flags for the last N days
GET	/api/dashboard	Everything the dashboard shows, in one request
GET	/api/analytics	Model evaluation results, archetype table and map sample

Table 10. The 19 API endpoints, read from `api/main.py`.

20.5 Local storage

Table	Columns	Holds
profile	id (always 1), data (JSON)	Name, sex, age, height, weight, activity, goal, diets, excluded ingredients, maximum cooking time
likes	recipe_id (primary key), added_at	Recipes the user liked; they drive recommendations
log	id, day, slot, recipe_id, servings, status, created_at	Every planned or eaten meal; status is 'planned' or 'eaten'

Table 11. SQLite schema (`data/user/nutrix.db`).

20.6 How recommendations are shown: match score and reasons

The ranker's probabilities are not calibrated for display, so the app shows a **match score** that blends two things a person cares about: how strongly the ranker expects them to cook the recipe (its rank among the candidates) and how well it fits one meal of their targets (calories near a third of the day, and macro balance near the target's). It is scaled to 60-99 and labelled as a match, not a probability. Each card also lists checked reasons: fits the calorie target, supports the protein goal, matches diet preferences, cooking time, and the liked recipe it most resembles by content similarity.

```
def _recommend_cards(p, n, course, exclude, courses, liked):
    t = targets_for(p)
    mask = mask_for(p).copy()
```

```

mask &= (catalog["course_final"] != "condiment").to_numpy()
if course:
    mask &= (catalog["course_final"] == course).to_numpy()
if courses:
    mask &= catalog["course_final"].isin(courses).to_numpy()
if exclude:
    mask[rec.index.known_positions(list(exclude))] = False
found = rec.recommend(liked, n=max(4 * n, 40), allowed=mask)
if found.empty:
    return []
ids = found["recipe_id"].to_numpy()
taste = found["score"].rank(pct=True).to_numpy()
match = 60 + 39 * (0.5 * taste + 0.5 * nutrition_fit(ids, t))
order = np.argsort(-match)[:n]
chosen = [int(ids[i]) for i in order]
explained = rec.explain(chosen, liked) if liked else None
cards = []
for k, i in enumerate(order):
    rid = chosen[k]
    because = None
    if explained is not None and pd.notna(explained.at[k, "because"]) and explained.at[k, "sim"]
        because = str(by_id.at[int(explained.at[k, "because"]), "display_name"])
    cards.append(**card(rid), "match": int(round(match[i])), "because": because, "reasons": r
return cards

```

20.7 The insight sentence

The dashboard's insight is rule-based, built on the meal log and the targets, and is labelled 'Insight', not AI. If meals are planned for the rest of the day the whole day is judged; otherwise what has been eaten is compared with what is normally eaten by that hour (breakfast by 10:00, lunch by 15:00, dinner by 21:00). A shortfall of more than 15% in a macronutrient triggers a suggestion of a recommended meal rich in it that fits the calories left.

20.8 The pages

Dashboard

Greeting and profile chips; today's calories and macros eaten against target with planned meals noted; nutrition balance radar (sugar and sodium against their limits); the insight; four top recommendations with photos, match scores and 'Add to Plan'; the reasons for the selected card; protein this week; days on target; and a banner.



Figure 19. Dashboard.

Recommendations

Up to 24 ranked recipes, filterable by course, each with its match score and the liked recipe it resembles. Hovering a card shows why it was picked.

NUTRIX
Intelligent Nutrition

Search for meals, ingredients or nutrition info...

Fri, 11 Sept 2026 2 Keep Going Guna

Recommendations
Ranked by a recommender trained on 221,732 Food.com reviews, then matched to your daily targets.

All Breakfast Main Side Snack Dessert Beverage

Baked Chicken Chimichangas
91% Match
Quick Meal
Because you liked Southwestern Quinoa Vegetable Casserole
467 kcal 30g Protein 48g Carbs 16g Fats
+ Add to Plan

Curried Red Lentil Dahl
90% Match
Vegetarian
Because you liked Southwestern Quinoa Vegetable Casserole
311 kcal 18g Protein 45g Carbs 6g Fats
+ Add to Plan

Breaded Chicken Fingers, Strips, Tenders...
89% Match
Rich & Hearty
Because you liked Mustard Grilled Chicken (Light)
1312 kcal 70g Protein 96g Carbs 69g Fats
+ Add to Plan

Pb & J Oatmeal
89% Match
Quick Meal
Because you liked Spinach Banana Smoothie
622 kcal 26g Protein 81g Carbs 23g Fats
+ Add to Plan

Baked Maple Oatmeal
88% Match
Starchy
326 kcal 12g Protein 39g Carbs 13g Fats
+ Add to Plan

Easy Huevos Rancheros
88% Match
Vegetarian • Quick Meal
Because you liked Southwestern Quinoa Vegetable Casserole
960 kcal 34g Protein 129g Carbs 33g Fats
+ Add to Plan

Lemon Pepper Fish Greek Style
87% Match
High Protein • Low Carb
Because you liked So Easy Salmon Patties
438 kcal 41g Protein 0g Carbs 29g Fats
+ Add to Plan

Deb's Favorite Way to Eat Fresh Fruit
87% Match
Quick Meal
Because you liked Spinach Banana Smoothie
192 kcal 2g Protein 21g Carbs 12g Fats
+ Add to Plan

Why this recommendation?
Baked Chicken Chimichangas - 91% match
○ Fits your calorie target
✓ Supports your protein goal
✓ No food restrictions set
✓ Ready in 30 minutes
✓ Similar to Southwestern Quinoa Vegetable Casserole, which you liked

How match works
Half is how strongly the ranker expects you to cook it, half is how well it fits one meal of your daily targets.
Browse all recipes →

"Better Food Brighter You"

Figure 20. Recommendations.

Food Library

Search across all 78,930 recipes by name or ingredient, filter by course and archetype, sort by reviews, rating, time, protein or calories, and restrict to the user's diet settings or to recipes with photos.

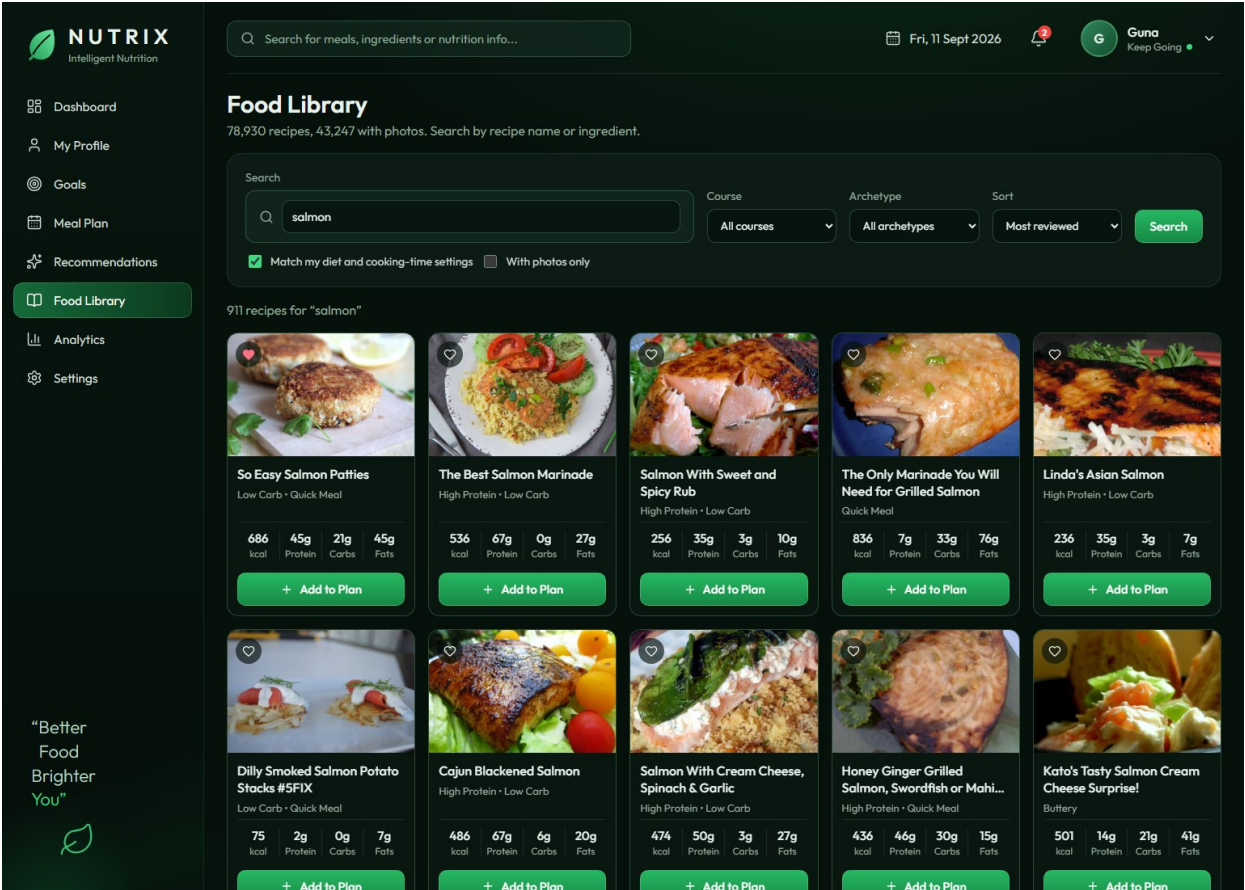


Figure 21. Food Library.

Recipe detail

Opened from any card: large photo, course, archetype, time, rating, description, ingredients, steps, nutrition per serving, like and add buttons, and six similar recipes found by content similarity.

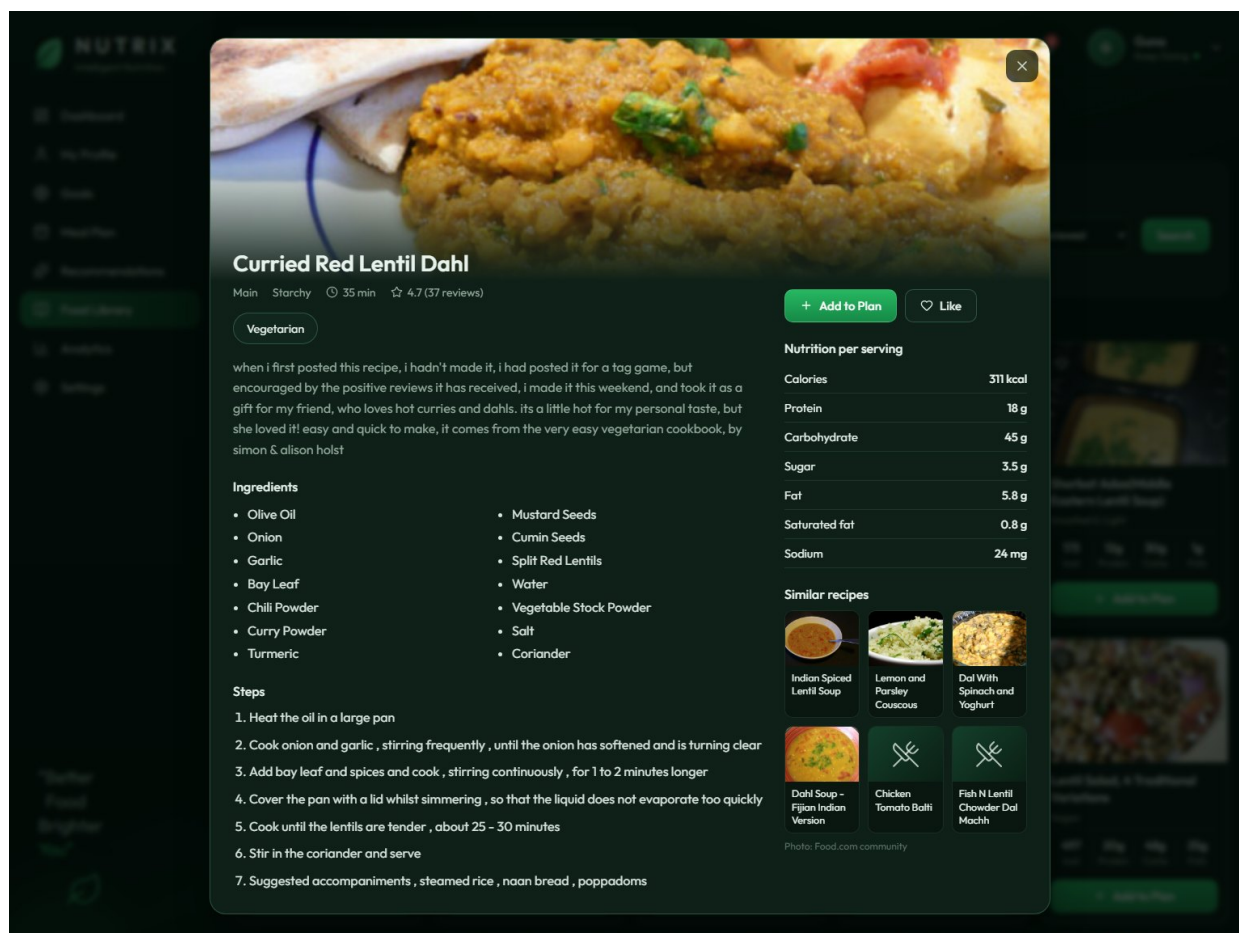


Figure 22. Recipe detail.

Meal Plan

Generate a 1, 3 or 7-day plan, shuffle it, and save it to the calendar. Below, each day's meals can be ticked off as eaten, removed, or browsed day by day.

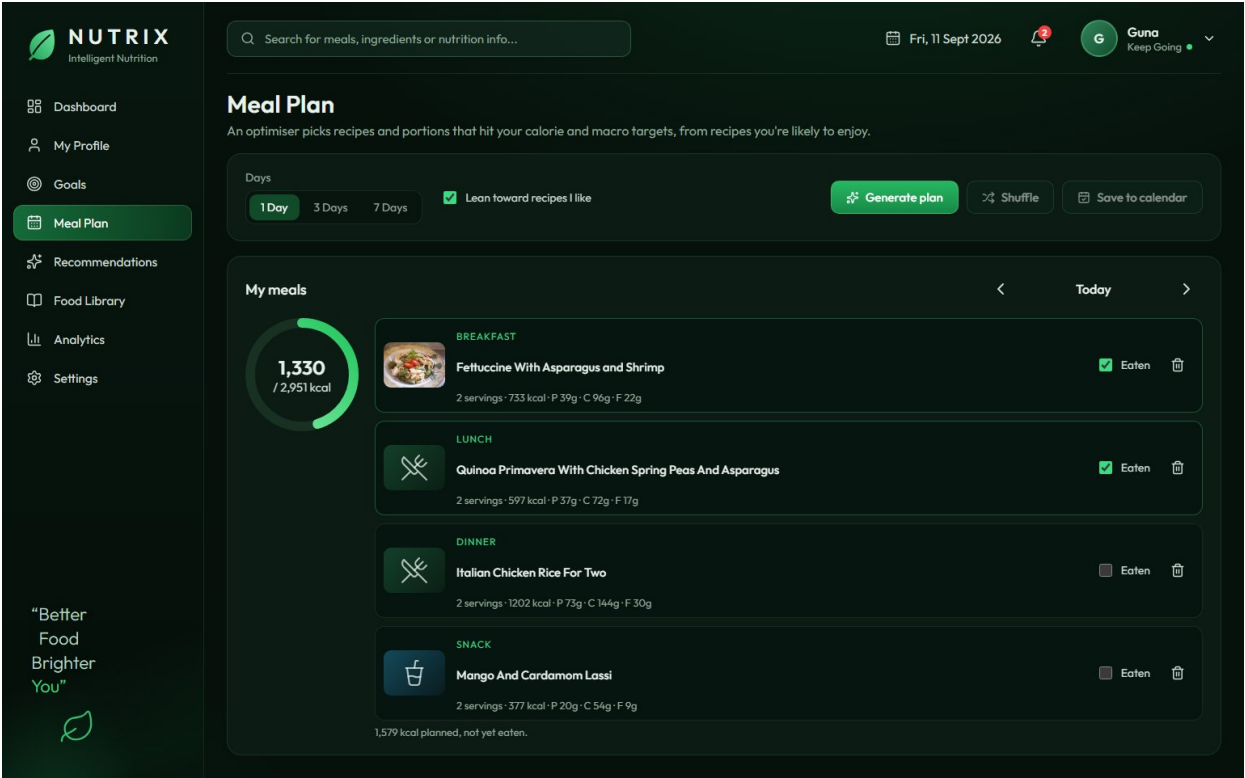


Figure 23. Meal Plan.

My Profile

Body measurements and activity, the resulting daily targets, and the liked recipes that drive recommendations.

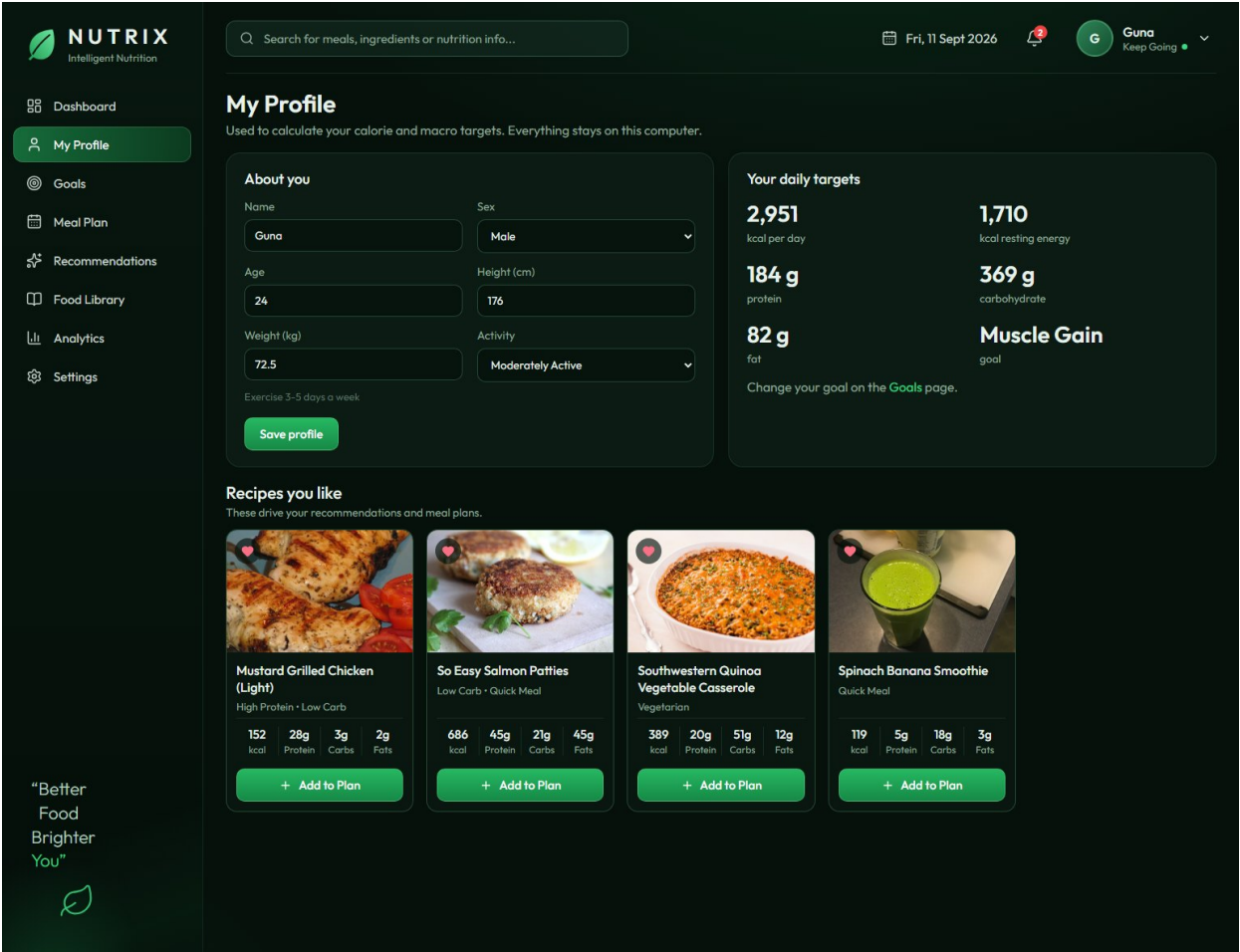


Figure 24. My Profile.

Goals

Choose weight loss, maintenance or muscle gain; see how resting energy, the activity multiplier and the goal adjustment make the target; macro split and limits; and what similar NHANES adults report eating.

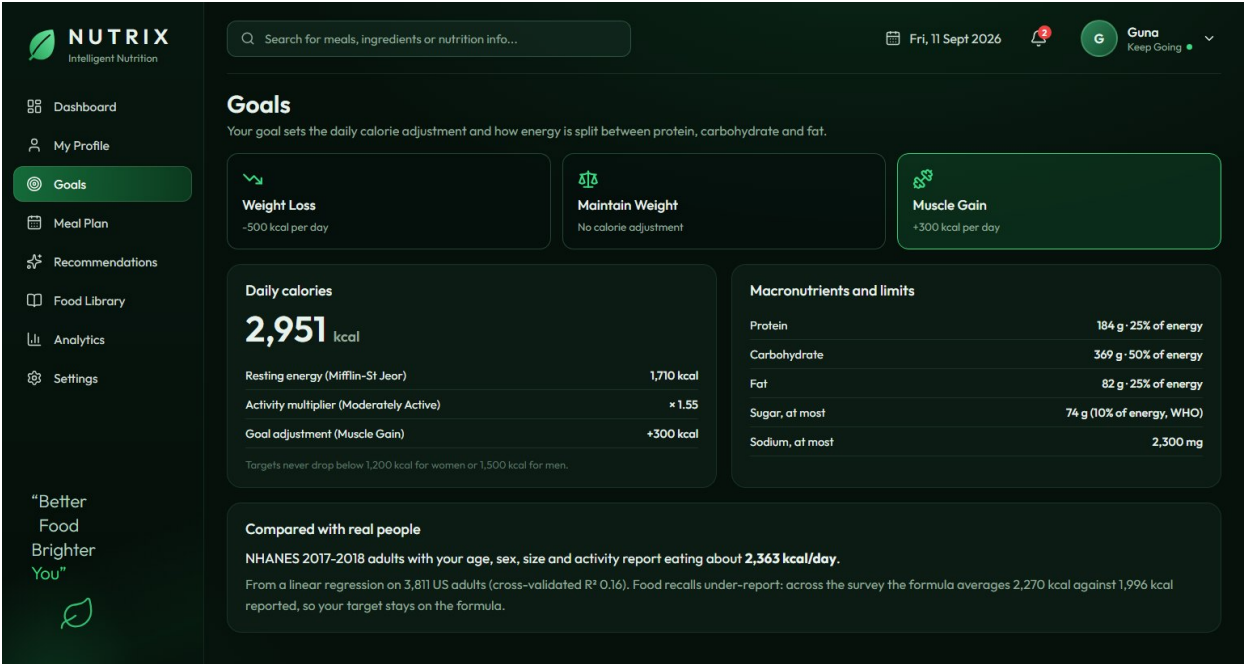


Figure 25. Goals.

Analytics

The last 30 days against target, the recommender's hit rate for every model, the other models' metrics, the interactive PCA recipe map and the archetype table.

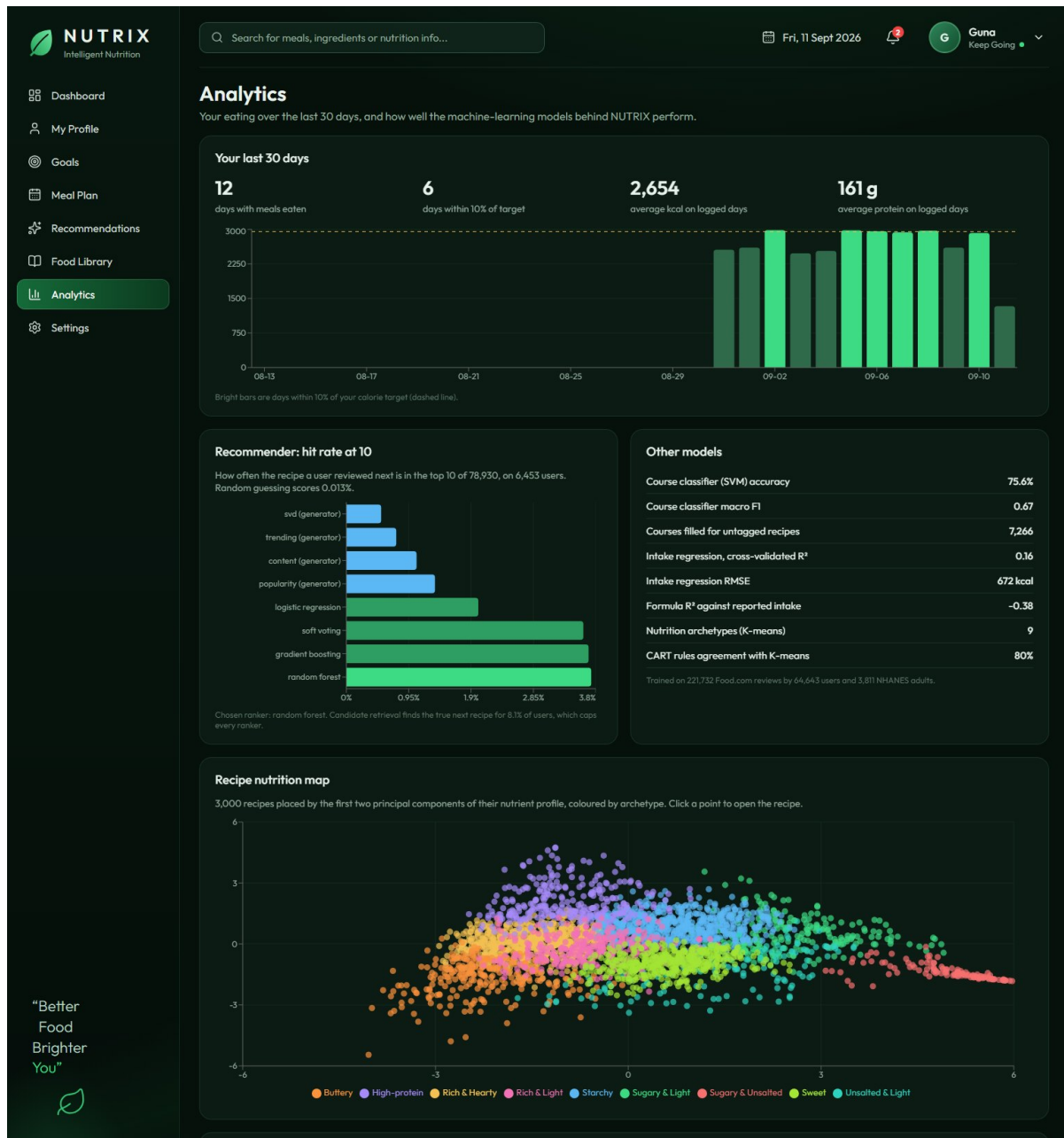


Figure 26. Analytics.

Settings

Diet filters, ingredients to leave out, maximum cooking time, clearing the log, and data credits.

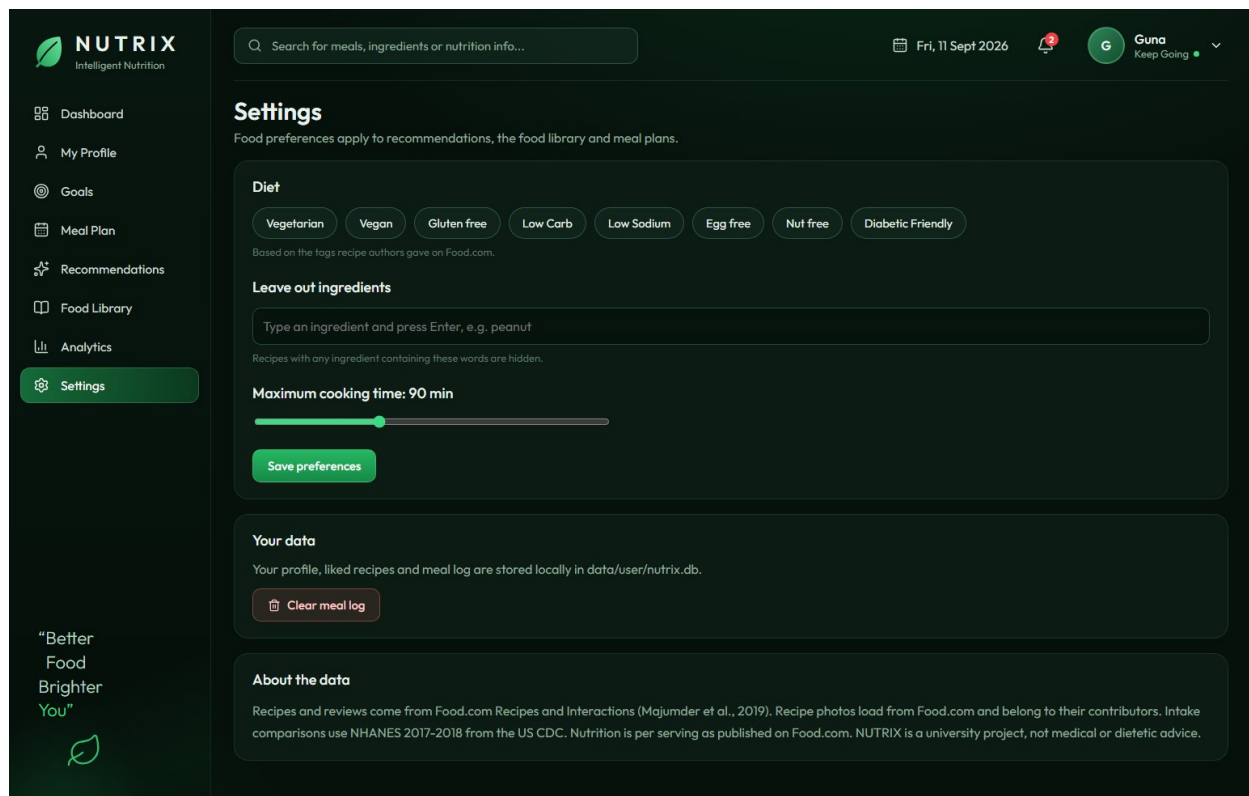


Figure 27. Settings.

Narrow screens

Below 1,100 pixels the sidebar collapses to icons, and below 720 pixels every grid stacks into one column, so the app works in a phone-sized browser window.

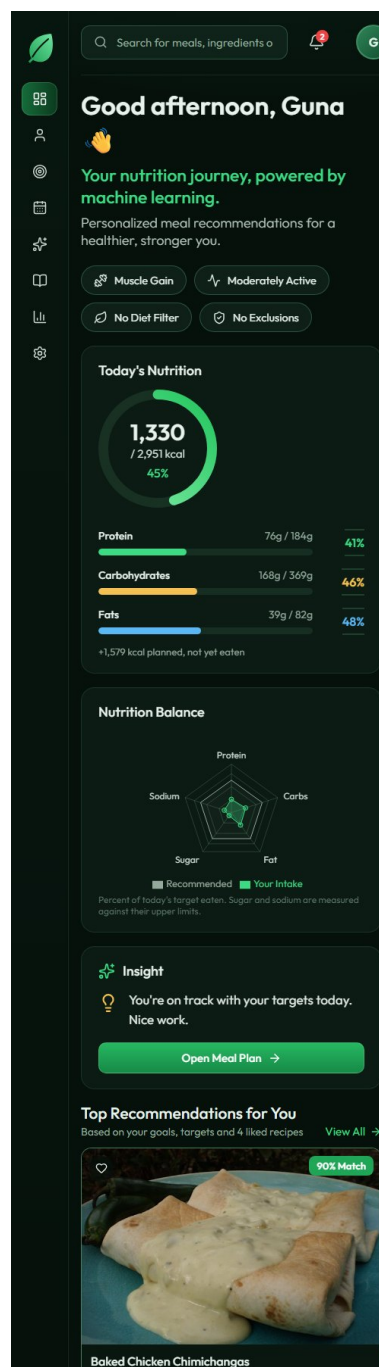


Figure 28. Dashboard in a narrow window.

21. Testing and Validation

21.1 API checks

`tests/check_api.py` runs every endpoint against a throwaway database through FastAPI's test client, checking the status code and a property of each response, including rejected invalid input. **24 of 24 checks passed.**

Check	Method	Path	Status	Time
catalogue metadata	GET	/api/meta	200	48 ms
default profile and targets	GET	/api/profile	200	95 ms
update profile	PUT	/api/profile	200	31 ms

Check	Method	Path	Status	Time
reject unknown diet	PUT	/api/profile	422	34 ms
reject out-of-range age	PUT	/api/profile	422	7 ms
recommendations without likes	GET	/api/recommendations	200	70 ms
search by ingredient	GET	/api/recipes	200	209 ms
recipe detail	GET	/api/recipes/304604	200	67 ms
missing recipe	GET	/api/recipes/999999999	404	6 ms
like a recipe	POST	/api/likes/304604	200	16 ms
list likes	GET	/api/likes	200	9 ms
recommendations after a like	GET	/api/recommendations	200	214 ms
log an eaten meal	POST	/api/log	200	23 ms
read today's log	GET	/api/log	200	13 ms
change servings	PATCH	/api/log/1	200	23 ms
reject invalid slot	POST	/api/log	422	5 ms
dashboard	GET	/api/dashboard	200	366 ms
generate a 3-day plan	POST	/api/plan	200	4044 ms
save plan to calendar	POST	/api/plan/save	200	26 ms
30-day progress	GET	/api/progress	200	29 ms
analytics	GET	/api/analytics	200	322 ms
delete a log entry	DELETE	/api/log/1	200	38 ms
unlike	DELETE	/api/likes/304604	200	26 ms
clear log	DELETE	/api/log	200	17 ms

Table 12. API checks.

21.2 Meal planner checks

`tests/check_planner.py` plans contrasting profiles and fails any plan more than 5% off its calorie target, more than 20% off a macronutrient target, or repeating a dish. **5 of 5 profiles passed.**

Profile	Days	Target	Time	Calorie gap	Macro gap	Result
Vegetarian woman, 25, maintain	7	2,030 kcal	8.0 s	0.5%	12.8%	pass
Man who likes curries, 25, maintain	7	2,301 kcal	9.4 s	2.2%	5.1%	pass
Vegan woman, 45, weight loss	3	1,200 kcal	3.1 s	0.6%	10.5%	pass
Active man, 22, muscle gain, no preference	7	3,580 kcal	21.2 s	1.7%	6.9%	pass
Active man, 22, muscle gain, rating preference	7	3,580 kcal	11.8 s	2.0%	13.1%	pass

Table 13. Planner checks.

21.3 Other checks

- **Notebook:** executed end to end with `nbconvert`; the checker confirms no cell raised an error.
- **Frontend:** `npm run build` runs the TypeScript compiler in strict mode before bundling, so type errors fail the build.

- **Visual review:** every page was captured with headless Chrome and compared with the reference design; the review caught charts that did not draw, a stretched logo and wrapping labels, which were fixed.
- **Data joins:** the photo join rate and name agreement were measured, which exposed the folder-split id bug.
- **Report:** every number in this document is read from the output files of the runs above.

22. Evaluation Summary

Model	Stage	HR@10	NDCG@10	MRR
svd	generator	0.53%	0.0032	0.0033
trending	generator	0.76%	0.0037	0.0038
content	generator	1.07%	0.0069	0.0069
popularity	generator	1.35%	0.0060	0.0056
logistic regression	ranker	2.01%	0.0119	0.0112
soft voting	ranker	3.61%	0.0221	0.0194
gradient boosting	ranker	3.69%	0.0217	0.0186
random forest	ranker	3.73%	0.0224	0.0194

Table 14. Recommendation on 6,453 users; random HR@10 is 0.013%.

Program	Model	Result
3	Intake linear regression	CV R^2 0.16; formula +313 kcal above reported intake
4.1	Bayesian logistic regression	ROC AUC 0.889; 4 of 7 weights exclude zero
4.2	Calibrated linear SVM	accuracy 75.6%, macro F1 0.67; 7,266 courses filled
5.1	K-means	k = 9 nutrition archetypes
5.2	Gaussian mixture	5 segments; 95.1% of users confidently assigned
5.3	Hierarchical clustering	baking, Italian-style, spice rub, stir-fry, soup-base groups
6	PCA / truncated SVD	65.5% of nutrient variance in 2 components; 26.8% of review variance in 64 factors
7	Categorical HMM	4 states, 3 persistent
8	CART surrogate	80.0% fidelity at depth 4
9	Random forest ranker	HR@10 3.73%, 2.8x the best generator
-	Meal planner	5 of 5 test profiles within 2.2% of calories and 13.1% of macros; a week takes 8-21 s

Table 15. Every other model.

23. Limitations

- Food.com nutrition comes from author-entered ingredients and serving counts. Cleaning removes inconsistent panels, but individual recipes can still be wrong.
- A review is a proxy for having cooked and liked a recipe. People who cook without reviewing are invisible, and very active reviewers dominate the collaborative signal.
- Leave-last-out evaluation fits generators on other users' full histories, including reviews after a target date. Serving targets on their own dates removes the most direct leakage, not all of it.
- The match score and insight are presentation logic on top of the models, not separately validated predictions.

- Photos cover 55% of recipes and belong to Food.com contributors; they are loaded for a university demonstration, not redistributed.
- The data is American, so tastes and portions may not transfer. Diet tags are authors' tags, not verified.
- The calorie target is a population equation and is not medical or dietetic advice.

24. Future Work

- **Better retrieval.** The ranker can only reorder what the generators find (8% recall). Item-item neighbours or a sequence-aware generator using the HMM phases are the obvious next step.
- **Learn from the app's own log.** Eaten meals are exactly the implicit feedback the recommender uses, so the log could fine-tune recommendations per user.
- **Deploy online.** The API can run on a cloud host and the built frontend on a static host, with accounts and a hosted database replacing the local profile and SQLite file.
- **Indian and regional recipes.** Adding datasets with local cuisines and portion sizes would make the recommendations fit Indian users better.
- **Micronutrients.** Joining recipes to USDA FoodData Central would add fibre and vitamins to the nutrition balance chart.

Part V — Reference

25. Reproducing Everything

```
cd C:\Users\Gunasekaran\Dev\nutrition-recommender
# 1. data (only once)
.venv\Scripts\python.exe src\download_data.py
# 2. train and evaluate every model the app uses (about three minutes)
.venv\Scripts\python.exe src\build.py
# 3. run all lab programs and regenerate the figures (about ten minutes)
.venv\Scripts\python.exe -m jupyter nbconvert --to notebook --execute --inplace ^
--ExecutePreprocessor.timeout=3600 notebooks\nutrition_ml_lab.ipynb
# 4. build the frontend once, then start NUTRIX and open http://localhost:8000
cd frontend
npm install
npm run build
cd ..
.venv\Scripts\python.exe -m uvicorn api.main:app --port 8000
# 5. development mode with hot reload: two terminals, then open http://localhost:5173
.venv\Scripts\python.exe -m uvicorn api.main:app --reload --port 8000
cd frontend && npm run dev
# 6. checks and this report
.venv\Scripts\python.exe tests\check_api.py
.venv\Scripts\python.exe tests\check_planner.py
.venv\Scripts\python.exe reports\build_report.py
```

Random seeds are fixed at 42. The planner's one-second solver limit means meal plans can differ slightly between runs on a busy machine; every other number reproduces exactly.

26. Glossary

Term	Meaning
Resting energy (BMR)	Energy used at rest per day, estimated by the Mifflin-St Jeor equation
PAL	Physical activity level, the multiplier from resting to total daily energy
Macronutrient	Protein, carbohydrate or fat
Atwater factors	4, 4 and 9 kcal per gram of protein, carbohydrate and fat
Percent daily value	A nutrient amount as a percentage of a reference daily amount
NHANES	US National Health and Nutrition Examination Survey
24-hour recall	An interview recording everything eaten the previous day
MET-minutes	Activity minutes weighted by intensity (METs)
Implicit feedback	Actions such as reviews that reveal interest without an explicit rating
Candidate generator	A cheap model that shortlists items from the whole catalogue
Ranker	A richer model that orders the shortlisted candidates
PureSVD	Collaborative filtering that scores items with a truncated SVD of the interaction matrix
TF-IDF	Term weighting that rewards words frequent in a document but rare across documents
Cold-start item	An item with no interaction history
Leave-last-out	Evaluation hiding each user's most recent item
HR@10	Share of users whose hidden item is ranked in the top ten
NDCG@10	Ranking metric that also rewards a higher position in the top ten
MRR	Mean of $1 / \text{rank}$ of the hidden item
Retrieval recall	Share of users whose hidden item is among the candidates at all
Negative sampling	Training a ranker on a sample of items the user did not choose
Calibration	Mapping model scores to probabilities that match observed frequencies
Macro F1	F1 score averaged equally over classes
Silhouette score	Cluster quality from -1 to 1 comparing within- and between-cluster distance
Hellinger transform	Square root of proportions, stabilising their variance
BIC	Bayesian Information Criterion, balancing fit against parameter count
Laplace approximation	Gaussian approximation of a posterior at its mode using the Hessian
Credible interval	Range containing a parameter with a stated posterior probability
Transition / emission matrix	HMM probabilities of moving between states / of observations from each state
Baum-Welch	EM algorithm that fits HMM parameters from sequences
Fidelity	Agreement between a surrogate model and the model it explains
Bagging / boosting	Averaging bootstrap models to reduce variance / sequentially correcting errors to reduce bias
Mixed-integer program	Optimisation with some whole-number variables
HiGHS	The open-source solver SciPy uses for linear and mixed-integer programs
API	Application programming interface; here JSON endpoints the frontend calls
FastAPI / Uvicorn	Python web framework for the API / the server that runs it
React / Vite	JavaScript library for the interface / tool that serves and builds it
SQLite	A database stored in a single local file

Term	Meaning
localhost	The address of the computer you are using; the app runs there
Smart App Control	Windows feature that blocks untrusted executable files

27. Expected Viva Questions

Why did you not continue with the original scripts?

Their nutrition values were random numbers, the 'healthy' label was a rule on the same features and the calorie target was generated from a linear formula. Every model result was therefore guaranteed and meaningless. The project keeps the idea and uses real data.

Why does the calorie formula get a negative R^2 , and why keep it?

R^2 is measured against reported intake, and the formula sits about 313 kcal above it because food recalls under-report. The formula estimates requirement, which is what a target should be, so it stays the target and the regression is shown as context.

Is an HR@10 of about 3.7% bad?

Random ranking scores 0.013%, so the ranker is about 295 times random and 2.8 times the best single generator. The ceiling is set by the data: 21% of next recipes had no earlier reviews and retrieval finds only 8.1%.

Why a two-stage recommender instead of just SVD?

No single signal is best: popularity and content each beat SVD on this sparse data, and they succeed on different users. A ranker combining them with recipe and user features more than doubles the best one.

How did you avoid leakage in evaluation?

Split by time per user; fit generators and rating features without the targets; serve each target on its own date; hide recipes not yet submitted; choose the ranker on validation users and use test targets only for final numbers.

What does the match percentage mean?

Half is the ranker's taste rank among candidates and half is nutrition fit for one meal of the user's targets, scaled to 60-99. It is a display score, not a probability.

Why calibrate the SVM?

SVM margins are not probabilities. Calibration lets the app assign a course only when the model is at least 50% confident, keeping doubtful recipes out of the planner.

Why keep nine archetypes when the silhouette is low?

Silhouette chose nine, and the low value honestly reflects continuous nutrition. The archetypes are still useful because their course mixes and nutrient centres differ clearly.

Why transform the shares before the Gaussian mixture?

Proportions near zero violate the Gaussian assumption, and components collapsed onto users who never cook an archetype, so BIC kept rising. The square-root transform and a covariance floor fixed it.

Why solve meal plans day by day with a time limit?

The whole week as one program found no feasible plan in 20 seconds. Per day, proving optimality takes seconds, so a one-second limit gives a fast plan and a guard re-solves any day more than 3% off calories.

Is NUTRIX a website or an application?

It is a web application that runs locally: a FastAPI server on your computer serves the React interface, which you open at <http://localhost:8000>. It is not hosted online, but it could be deployed.

How were the photos matched?

Photo URLs in a larger Food.com scrape contain the recipe id split into folders. Joining the folders gives the id used by the main dataset, matching 55% of recipes. The join rate and name agreement were checked, which caught an early bug.

What would you do next?

Improve candidate retrieval, learn from the app's own meal log, add regional cuisines and micronutrients, and deploy it online with accounts.

28. References

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Centers for Disease Control and Prevention. National Health and Nutrition Examination Survey 2017-2018 data files.
- Cremonesi, P., Koren, Y. and Turrin, R. (2010). Performance of recommender algorithms on top-N recommendation tasks. *Proceedings of the 4th ACM Conference on Recommender Systems*, 39-46.
- Huangfu, Q. and Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10, 119-142.
- Karo8870 (2025). food.com-parsed-dataset. Hugging Face datasets.
- Majumder, B. P., Li, S., Ni, J. and McAuley, J. (2019). Generating personalized recipes from historical user preferences. *Proceedings of EMNLP-IJCNLP 2019*.
- Mifflin, M. D., St Jeor, S. T., Hill, L. A., Scott, B. J., Daugherty, S. A. and Koh, Y. O. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition*, 51(2), 241-247.
- Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257-286.
- Willett, W. (2012). *Nutritional Epidemiology* (3rd ed.). Oxford University Press.
- World Health Organization (2015). *Guideline: Sugars intake for adults and children*.